

Hello, HTML!

Xin chào, HTML!

Building Your Very First Webpage

Xây dựng trang web đầu tiên của bạn



HTML (Structure)

HTML (Cấu trúc)

Building the skeleton — pillars, walls, and window frames.

Dựng bộ khung — cột, tường và khung cửa sổ.



CSS (Design)

CSS (Thiết kế)

Interior design — painting walls and choosing wallpapers.

Trang trí nội thất — sơn tường, chọn giấy dán tường.



JavaScript (Function)

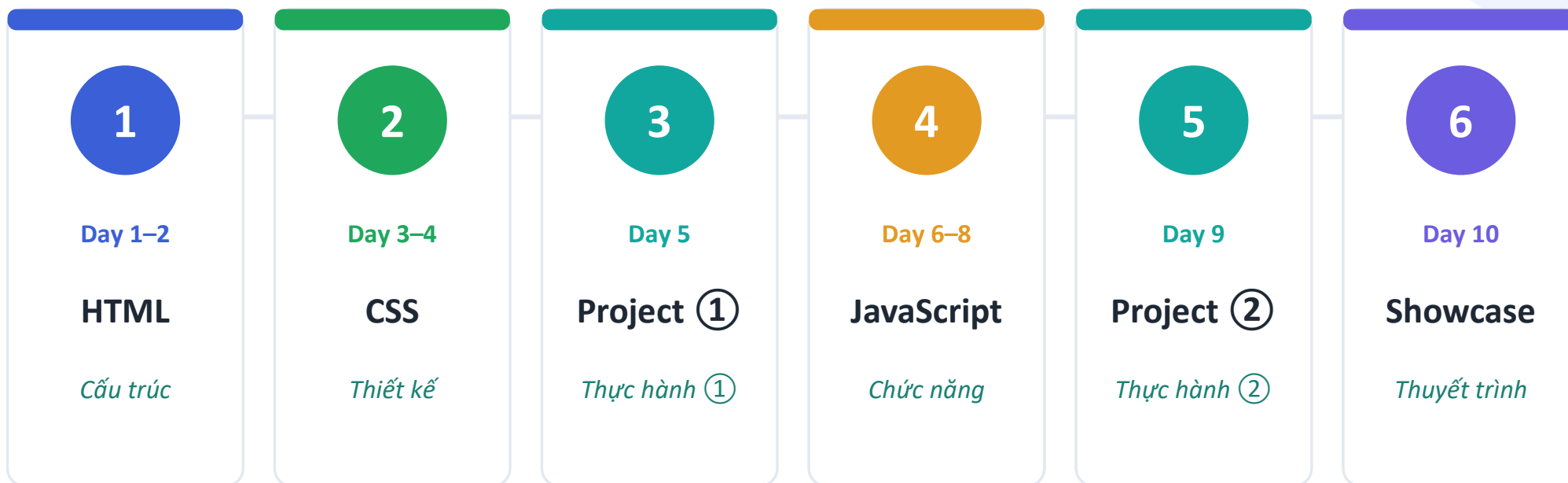
JavaScript (Chức năng)

Adding features — elevators and automatic doors.

Thêm tính năng — thang máy và cửa tự động.

Our 10-Day Roadmap

Lộ trình 10 ngày của chúng ta



HTML → CSS → JavaScript · two hands-on team projects along the way

HTML → CSS → JavaScript, kèm hai dự án nhóm thực hành trên hành trình.

Daily Schedule · Days 1–5

Thời khóa biểu · Ngày 1–5

DAY 1	Orientation & HTML Basics <i>Định hướng & HTML cơ bản</i> Setup (VS Code, browser); HTML structure & semantic tags <i>Cài đặt; cấu trúc HTML & thẻ ngữ nghĩa</i>	5h
DAY 2	HTML In-Depth <i>HTML nâng cao</i> Forms (input, label, button); tables & multimedia; build a page <i>Biểu mẫu; bảng & đa phương tiện; hoàn thiện trang</i>	5h
DAY 3	CSS Basics <i>CSS cơ bản</i> CSS linking, selectors, box model; color/font/background styling <i>Liên kết CSS, bộ chọn, box model; tạo kiểu</i>	5h
DAY 4	CSS Layout <i>Bố cục CSS</i> Flexbox & alignment; media queries (responsive layout) <i>Flexbox & căn chỉnh; media query (responsive)</i>	5h
DAY 5	Mini Project ① <i>Dự án nhỏ ①</i> Build a self-introduction webpage (HTML + CSS); team feedback <i>Trang giới thiệu bản thân (HTML + CSS); phản hồi nhóm</i>	3h

Daily Schedule · Days 6–10

Thời khóa biểu · Ngày 6–10

DAY 6	JavaScript Basics <i>JavaScript cơ bản</i> JS linking, variables, data types, operators; functions & if/else <i>Liên kết JS, biến, kiểu dữ liệu; hàm & câu điều kiện</i>	5h
DAY 7	JavaScript Events <i>Sự kiện JavaScript</i> for loops, arrays; DOM selection & event handlers (click/input) <i>Vòng lặp, mảng; chọn DOM & xử lý sự kiện</i>	5h
DAY 8	JavaScript DOM <i>Thao tác DOM</i> Add / edit / remove elements; To-do or quiz mini app <i>Thêm / sửa / xóa phần tử; ứng dụng nhỏ To-do/quiz</i>	5h
DAY 9	Mini Project ② <i>Dự án nhỏ ②</i> Team free-topic: plan & HTML; CSS + JS features; polish <i>Nhóm tự chọn chủ đề: kế hoạch, HTML, CSS + JS; hoàn thiện</i>	5h
DAY 10	Presentation & Wrap-up <i>Thuyết trình & Tổng kết</i> Team presentations; course retrospective & completion <i>Thuyết trình nhóm; nhìn lại khóa học & hoàn thành</i>	3h

What is HTML?

HTML là gì?

HyperText

Markup

Language

Ngôn ngữ đánh dấu siêu văn bản

"The language that tells the computer how to structure a webpage."

"Ngôn ngữ cho máy tính biết cách cấu trúc một trang web."

It tells the computer: **"This is a heading!"** or **"This is an image!"**

Nó nói với máy tính: "Đây là tiêu đề!" hoặc "Đây là hình ảnh!"

We use **'Tags'** to communicate with the computer.

Chúng ta dùng 'Thẻ' (Tags) để giao tiếp với máy tính.



```
<heading> Hello, Danang!  
</heading>
```

The Magic Tool: Tags

Công cụ kỳ diệu: Thẻ (Tags)

`<h1>`

Opening Tag

Thẻ mở

Content

Content

Nội dung

`</h1>`

Closing Tag

Thẻ đóng

Most tags come in pairs: Opening and Closing!

Hầu hết các thẻ đi theo cặp: thẻ mở và thẻ đóng!

Don't forget the slash (/) in the closing tag!

Đừng quên dấu gạch chéo (/) trong thẻ đóng!

The Blueprint: Basic Structure

Bản thiết kế: Cấu trúc cơ bản

```
<!DOCTYPE html>
<html>
  <head>
    <!-- Secret Info Room -->
  </head>
  <body>
    <!-- Visible Content Room -->
  </body>
</html>
```

❶ **<!DOCTYPE html>**

Declares: "This is an HTML5 document!"

Khai báo: 'Đây là tài liệu HTML5!'

❷ **<html>**

The root container for all HTML elements.

Vùng chứa gốc cho mọi phần tử HTML.

❸ **<head> (The Head)**

Settings for the browser (title, language, etc.).

Cài đặt cho trình duyệt (tiêu đề, ngôn ngữ, v.v.).

❹ **<body> (The Body)**

Everything visible to the user goes here.

Mọi thứ người dùng nhìn thấy nằm ở đây.

<head>: The Secret Diary

<head>: Cuốn nhật ký bí mật

"Hidden Settings" · Cài đặt ẩn

Information that the browser needs, but isn't shown directly on the webpage.

Thông tin trình duyệt cần, nhưng không hiển thị trực tiếp trên trang web.

- **<title>:** The name on the browser tab
tên trên tab trình duyệt
- **<meta charset="UTF-8">:** Prevents text errors
ngăn lỗi hiển thị chữ
- Links to CSS, SEO info, and more
liên kết CSS, thông tin SEO, và hơn thế



```
<head>
  <!-- Encoding setting -->
  <meta charset="UTF-8">
  <!-- Webpage Title -->
  <title>My First Webpage</title>
</head>
```

My First Webpage

Changing <title> changes the tab name!

Thay đổi <title> sẽ đổi tên tab!

<body>: The Real World

<body>: Thế giới thực

- Everything visible to the user
Mọi thứ người dùng nhìn thấy
- Text, Images, Buttons, Videos, etc.
Văn bản, hình ảnh, nút, video, v.v.
- Where most of your code will live
Nơi phần lớn mã của bạn sẽ nằm



```
<body>
  Hello! Everything you see
  on the screen goes here.
</body>
```



Browser View · Giao diện trình duyệt

This area is the body!

Vùng này chính là body!

Whatever you type here appears instantly for users.

Bất cứ gì bạn gõ ở đây sẽ hiện ngay cho người dùng.

Headings and Paragraphs

Tiêu đề và Đoạn văn

HTML Code



```
<h1>Main Title</h1>
<h2>Sub Title</h2>
<h3>Section Title</h3>
<p>This is a paragraph.
It adds spacing and line breaks
automatically!</p>
```

Browser Preview

Main Title

Sub Title

Section Title

This is a paragraph. It adds spacing and line breaks automatically!

<h1> to <h6>: Headings. Smaller numbers mean larger text. Use <h1> for the most important title.

Thẻ tiêu đề. Số nhỏ hơn = chữ lớn hơn. Dùng <h1> cho tiêu đề quan trọng nhất.

<p>: Paragraph. Used for regular text blocks. It creates vertical space between paragraphs.

Đoạn văn. Dùng cho khối văn bản thường. Tạo khoảng cách dọc giữa các đoạn.

The Link: <a> Tag

Liên kết: Thẻ <a>

```
<a href="URL">Clickable Text</a>
```

<a> : Anchor · Mỏ neo

Just like an anchor holds a ship, this tag connects your page to another destination.

Giống như mỏ neo giữ con tàu, thẻ này kết nối trang của bạn tới một đích khác.

href : Attribute · Thuộc tính

Short for Hypertext Reference. It tells the computer exactly where to go.

Viết tắt của Hypertext Reference. Cho máy tính biết chính xác nơi cần đến.

Practice Tip: Try `<a href="https://www.google.com">Go to Google</a>` in your code!

Mẹo thực hành: Thử `<a href="...">Go to Google</a>` trong mã của bạn!

Visuals: Tag

Hình ảnh: Thẻ

```

```

src : Source · Nguồn

The path to your image. It can be a web link or a file name on your computer.

Đường dẫn tới hình ảnh. Có thể là liên kết web hoặc tên tệp trên máy của bạn.

alt : Alternative · Thay thế

Text that appears if the image fails to load. Also used for accessibility.

Văn bản hiện khi ảnh không tải được. Cũng dùng cho khả năng tiếp cận.

⚠ The tag is a Self-Closing tag (Thẻ tự đóng)!

It has no closing tag like — it starts and ends in one go! · Không có thẻ đóng như — bắt đầu và kết thúc trong một lần!

Organization: and Tags

Sắp xếp: Thẻ và

HTML Code

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Grape</li>
</ul>
```

Browser Preview

- Apple
- Banana
- Grape

** : Unordered List** · Danh sách không thứ tự

The 'container' for a list without numbers. Automatically adds bullet points (•).

'Vùng chứa' cho danh sách không đánh số. Tự động thêm dấu chấm đầu dòng (•).

** : List Item** · Mục danh sách

Represents each individual 'item'. It must always be inside a .

Đại diện cho từng 'mục'. Luôn phải nằm trong .

[Practice] My First Webpage!

[Thực hành] Trang web đầu tiên của tôi!

1

Your Name · Tên của bạn

Use the `<h1>` tag to write your name as the main title.

Dùng thẻ `<h1>` để viết tên bạn làm tiêu đề chính.

2

Introduction · Giới thiệu

Use the `<p>` tag to write a sentence about your hobbies.

Dùng thẻ `<p>` để viết một câu về sở thích của bạn.

3

Add a Photo · Thêm ảnh

Use the `<img>` tag with a `src` attribute to display a picture.

Dùng thẻ `<img>` với thuộc tính `src` để hiển thị ảnh.

4

Link it Up · Thêm liên kết

Use the `<a>` tag to link to your favorite website.

Dùng thẻ `<a>` để liên kết tới trang web yêu thích.

Coding Hint (Write inside the `<body>!` · Viết bên trong `<body>!`)



```
<h1>John Doe's Page</h1>
<p>I love playing soccer and coding.</p>

<a href="url">Go to YouTube</a>
```

Summary & Next Steps

Tóm tắt và Bước tiếp theo

Key Takeaways · Điểm chính

- **HTML is the skeleton of a website!**
HTML là bộ khung của một trang web!
- **Tags usually come in pairs (e.g., `<h1>...</h1>`)**
Thẻ thường đi theo cặp (vd: `<h1>...</h1>`)
- **`<head>` is for settings, `<body>` is for content!**
`<head>` dùng cho cài đặt, `<body>` dùng cho nội dung!

What's Next? · Tiếp theo là gì?

Advanced HTML · HTML nâng cao

- › Creating cool input fields and buttons
Tạo ô nhập liệu và nút bấm thú vị
- › Handling images and videos like a pro
Xử lý hình ảnh và video chuyên nghiệp
- › Making your webpage more interactive
Làm trang web tương tác hơn

Great job today! See you next time! · Làm tốt lắm hôm nay! Hẹn gặp lại lần sau!

INTRODUCTION TO HTML · GIỚI THIỆU VỀ HTML

Day 2: Forms, Tables, and Multimedia

Ngày 2: Biểu mẫu, Bảng và Đa phương tiện

HTML Deep Dive · Tìm hiểu sâu về HTML

What We Covered Yesterday

Những gì chúng ta đã học hôm qua



HTML Boilerplate

Khung HTML

`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>` — the skeleton of every page

bộ khung của mọi trang



DOM Tree & Nesting

Cây DOM & Lồng nhau

Parent / Child / Sibling relationships; FILO nesting rules

Quan hệ cha / con / anh em; quy tắc lồng FILO



Block vs. Inline

Khối và Nội tuyến

`<div>`, `<p>`, `<h1>`—`<h6>` vs. `<span>`, `<a>`, `<strong>`, `<em>`

phần tử khối và nội tuyến



Attributes & IDs

Thuộc tính & ID

`name="value"` pairs; unique id vs. shared class

cặp `name="value"`; id duy nhất và class dùng chung



Essential Tags

Thẻ thiết yếu

Headings, paragraphs, links, ul/ol lists

Tiêu đề, đoạn văn, liên kết, danh sách ul/ol



Semantic HTML5

HTML5 ngữ nghĩa

`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`

các thẻ ngữ nghĩa

TODAY

What We'll Learn Today

Hôm nay chúng ta sẽ học gì

01

HTML Forms

Biểu mẫu HTML

Collect user input — text fields, buttons, choices.

Thu thập dữ liệu người dùng — ô nhập, nút, lựa chọn.

02

HTML Tables

Bảng HTML

Organize data in neat rows and columns.

Sắp xếp dữ liệu thành hàng và cột gọn gàng.

03

Multimedia

Đa phương tiện

Add images, audio and video to your page.

Thêm hình ảnh, âm thanh và video vào trang.

HTML Forms

Biểu mẫu HTML

What is a Form? · Biểu mẫu là gì?

Forms collect user input and send it to a server.

Biểu mẫu thu thập dữ liệu nhập của người dùng và gửi tới máy chủ.

- Every interactive website uses forms
Mọi website tương tác đều dùng biểu mẫu
- Login pages, search bars, contact forms
Trang đăng nhập, thanh tìm kiếm, biểu mẫu liên hệ
- Data travels via HTTP (GET or POST)
Dữ liệu truyền qua HTTP (GET hoặc POST)

The Basic Structure · Cấu trúc cơ bản



```
<form>
  <label for="name">Your Name:</label>
  <input type="text"
    id="name"
    name="name">
  <button type="submit">Submit</button>
</form>
```

Anatomy of a Form

Giải phẫu một biểu mẫu



```
<form action="/save" method="post">
  <label for="email">Email:</label>
  <input type="email" id="email">
  <button>Send</button>
</form>
```

● **<form>**

The wrapper that groups all controls.

Vùng bao nhóm mọi điều khiển.

● **<label>**

Describes what an input is for.

Mô tả ô nhập dùng để làm gì.

● **<input>**

Where the user types or chooses.

Nơi người dùng gõ hoặc chọn.

● **<button>**

Submits the form's data.

Gửi dữ liệu của biểu mẫu.

How they nest · Cách lồng nhau

<form>

<label>

<input>

<button>

The <label> Tag

Thẻ <label>

A label names an input. Click the label and the input focuses — great for usability and accessibility.

Nhãn đặt tên cho ô nhập. Nhấp vào nhãn thì ô nhập được chọn — tốt cho khả năng dùng và tiếp cận.

✓ **Linked with for / id**

```
<label for="city">City</label>
<input id="city">
// for matches id
```

✗ **No connection**

```
City
<input>
// label not linked
```

Tip: always pair a <label> with its <input> using for="id".

Mẹo: luôn ghép <label> với <input> bằng for="id".

SECTION 01

Input Types

Các loại Input

`type="text"`

Single-line text entry

Nhập văn bản một dòng

`type="password"`

Hides characters as you type

Ẩn ký tự khi bạn gõ

`type="email"`

Validates email format

Kiểm tra định dạng email

`type="number"`

Accepts numeric values only

Chỉ nhận giá trị số

`type="checkbox"`

Select multiple options

Chọn nhiều tùy chọn

`type="radio"`

Select one from a group

Chọn một trong nhóm

`type="date"`

Date picker widget

Tiện ích chọn ngày

`type="file"`

Upload a file

Tải lên một tệp

More Form Controls

Thêm các điều khiển biểu mẫu

<textarea>

Multi-line text · Văn bản nhiều dòng

```
<textarea rows="4">
</textarea>
```

For long messages & comments.

Cho tin nhắn & bình luận dài.

<select>

Drop-down menu · Menu thả xuống

```
<select>
  <option>Hue</option>
  <option>Da Nang</option>
</select>
```

Pick one from a list.

Chọn một mục từ danh sách.

SECTION 01

Useful Form Attributes

Các thuộc tính hữu ích

action

URL where the data is sent

URL nơi dữ liệu được gửi đến

method

GET (in URL) or POST (hidden body)

GET (trên URL) hoặc POST (ẩn trong nội dung)

placeholder

Hint text shown inside an empty field

Văn bản gợi ý hiện trong ô trống

required

Field must be filled before submit

Bắt buộc nhập trước khi gửi

value

A default starting value

Giá trị mặc định ban đầu

Example · Ví dụ



```
<form action="/save"
      method="post">
  <input
    placeholder="Name"
    required>
</form>
```


SECTION 01

Putting It Together

Ghép tất cả lại

A simple contact form using everything we've learned.

Một biểu mẫu liên hệ đơn giản dùng mọi thứ đã học.



```
<form action="/contact" method="post">
  <label for="n">Name</label>
  <input id="n" type="text" required>
  <label for="e">Email</label>
  <input id="e" type="email">
  <textarea placeholder="Message"></textarea>
  <button type="submit">Send</button>
</form>
```

Preview · Xem trước

Name

Email

Message

Send

SECTION 02

HTML Tables

Bảng HTML

<table>

The outer wrapper — contains all rows and cells

Vùng bao ngoài — chứa mọi hàng và ô

<tbody>

Body section — wraps all data rows

Phần thân — bao mọi hàng dữ liệu

<th>

Table Header — bold, centered by default

Ô tiêu đề — in đậm, căn giữa mặc định

<thead>

Header section — wraps the title row

Phần đầu — bao hàng tiêu đề

<tr>

Table Row — a horizontal row of cells

Hàng của bảng — một hàng ngang gồm các ô

<td>

Table Data — a regular data cell

Ô dữ liệu — một ô dữ liệu thường

Example · Ví dụ



```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>24</td>
    </tr>
  </tbody>
</table>
```

Build a Table Step by Step

Dựng bảng từng bước

1

Start with `<table>` as the container.

Bắt đầu bằng `<table>` làm vùng chứa.

2

Add rows with `<tr>`.

Thêm hàng bằng `<tr>`.

3

Header cells use `<th>`; data cells use `<td>`.

Ô tiêu đề dùng `<th>`; ô dữ liệu dùng `<td>`.

4

Group with `<thead>` and `<tbody>`.

Nhóm bằng `<thead>` và `<tbody>`.

Rendered result · Kết quả hiển thị

Name	Age	City
Alice	24	Hue
Bao	22	Da Nang
Chi	23	Hoi An

Spanning Cells

Ô gộp — *colspan* & *rowspan*

Merge cells across columns or rows to build richer tables.

Gộp ô theo cột hoặc hàng để tạo bảng phong phú hơn.

colspan — merge across columns · gộp theo cột

colspan="2"	
A	B
C	D

```
<td colspan="2">Title</td>
```

rowspan — merge across rows · gộp theo hàng

rowspan="2"	E
	F
G	

```
<td rowspan="2">Side</td>
```

SECTION 03

Images —

Hình ảnh —

The tag embeds a picture. It is self-closing — no needed.

Thẻ nhúng một bức ảnh. Nó tự đóng — không cần .



```

```

Always write alt text — it helps accessibility & SEO.

Luôn viết alt — giúp khả năng tiếp cận & SEO.

src

path to the image file

đường dẫn tới tệp ảnh

alt

text shown if the image fails / for screen readers

văn bản hiện khi ảnh lỗi / cho trình đọc màn hình

width / height

size in pixels

kích thước theo pixel

SECTION 03

Audio & Video

Âm thanh & Video

<audio>

Embed sound · Nhúng âm thanh

```
<audio controls>
  <source src="song.mp3"
    type="audio/mpeg">
</audio>
```

controls — shows play bar

hiện thanh phát

<video>

Embed a video · Nhúng một video

```
<video width="640" controls
  autoplay loop>
  <source src="clip.mp4">
</video>
```

controls — play/pause bar

thanh phát/dừng

autoplay — plays automatically

tự động phát

loop — repeats

lặp lại

Best Practices

Thực hành tốt & Khả năng tiếp cận



Label every input

Gắn nhãn mọi ô nhập

Use `<label for>` so forms are usable by everyone.

Dùng `<label for>` để ai cũng dùng được biểu mẫu.



Always add alt text

Luôn thêm alt

Describe images for screen readers.

Mô tả ảnh cho trình đọc màn hình.



Use `<th>` for headers

Dùng `<th>` cho tiêu đề

Table headers help readers understand data.

Tiêu đề bảng giúp người đọc hiểu dữ liệu.



Keep structure clean

Giữ cấu trúc gọn

Indent and close every tag you open.

Thụt lề và đóng mọi thẻ đã mở.

Hands-on Practice

Thực hành

Build a simple Student Profile Page using today's tags · Xây dựng trang Hồ sơ Sinh viên bằng các thẻ hôm nay

01

Create the file

Tạo tệp

Open VS Code. Create profile.html and add the HTML boilerplate.

Mở VS Code. Tạo profile.html và thêm khung HTML.

02

Add a Table

Thêm bảng

Build a 3-row table showing Name, Age, and Hometown.

Tạo bảng 3 hàng: Tên, Tuổi, Quê quán.

03

Add an Image

Thêm ảnh

Insert an tag. Use any image URL you like.

Chèn thẻ . Dùng URL ảnh bất kỳ.

04

Add a Contact Form

Thêm biểu mẫu

Create a form with name, email fields and a Submit button.

Tạo biểu mẫu có ô tên, email và nút Gửi.

05

Open in Chrome

Mở trong Chrome

Double-click profile.html to preview your page!

Nhấp đúp profile.html để xem trước trang!

AVOID THESE

Common Mistakes & Tips

Lỗi thường gặp & Mẹo

⚠ Common mistakes

- ✗ **Forgetting to close a tag**
Quên đóng một thẻ
- ✗ **No name on inputs — data won't send**
Thiếu name — dữ liệu không gửi
- ✗ **Missing alt on images**
Thiếu alt trên ảnh
- ✗ **Mixing up `<th>` and `<td>`**
Nhầm `<th>` và `<td>`

✓ Helpful tips

- ✓ **Indent nested tags for clarity**
Thụt lề thẻ lồng cho rõ ràng
- ✓ **Test the form in the browser**
Thử biểu mẫu trên trình duyệt
- ✓ **Use semantic, meaningful ids**
Dùng id có ý nghĩa
- ✓ **Preview often as you build**
Xem trước thường xuyên khi làm

REVIEW

Check Yourself

Tự kiểm tra

Can you answer these without looking back?

Bạn có thể trả lời mà không cần xem lại không?

1

Which tag wraps all form controls?

Thẻ nào bao mọi điều khiển biểu mẫu?

2

What links a `<label>` to its `<input>`?

Cái gì liên kết `<label>` với `<input>`?

3

Which tag makes a table header cell?

Thẻ nào tạo ô tiêu đề bảng?

4

What attribute describes an image?

Thuộc tính nào mô tả một ảnh?

5

How do you merge two columns in a table?

Làm sao gộp hai cột trong bảng?

SUMMARY

Day 2 Complete!

Hoàn thành Ngày 2!



Forms · *Biểu mẫu*

`<form>, <label>, <input>, <textarea>, <select>, <button>`



Input Types · *Các loại input*

`text, password, email, number, checkbox, radio, date, file`



Tables · *Bảng*

`<table>, <thead>, <tbody>, <tr>, <th>, <td>, colspan, rowspan`



Multimedia · *Đa phương tiện*

`<img src alt>, <audio>, <video controls>`



Good habits · *Thói quen tốt*

`labels, alt text, clean structure, accessibility`

Great job — keep building! · Làm tốt lắm — tiếp tục xây dựng nhé!

INTRODUCTION TO CSS

CSS BASICS — DAY 3

Connection Methods · Selectors · Box Model · CSS Styling

CSS Cơ bản – Ngày 3: cách gắn CSS vào HTML, bộ chọn (selector), mô hình hộp (box model) và tạo kiểu cho trang web.

What We Covered Yesterday

Ôn tập hôm qua



HTML Forms

Form, Input, Label, Button

Các thẻ tạo biểu mẫu nhập liệu



Tables

Display structured data

Hiển thị dữ liệu có cấu trúc



Multimedia Tags

Image and Video Elements

Phần tử hình ảnh và video



HTML Page Project

Complete a simple webpage

Hoàn thành một trang web đơn giản

Table of Contents

Mục lục

01

CSS Connection

Inline, Internal, and External CSS linking techniques

02

CSS Selectors

Element, Class, ID, and Combining selectors

03

Box Model

Content, Padding, Border, Margin, and Box-sizing

04

CSS Styling

Color, Font, Background-Color Styling

05

Hands-on Practice

Apply CSS to HTML pages with practical exercises

1) Gắn CSS 2) Bộ chọn 3) Mô hình hộp 4) Tạo kiểu 5) Thực hành

Morning Session Overview

01

CSS Connection Methods

Inline · Internal · External

02

CSS Selectors

Element · ID · Class · Combining

03

Box Model

Content · Padding · Border · Margin

Buổi sáng: cách gắn CSS vào HTML, các bộ chọn phần tử, và mô hình hộp (box model).

CSS Connection Methods

Các cách gắn CSS vào HTML



Inline CSS

Add CSS directly to HTML elements using the style attribute

```
<p style="color:red;">
  Hello
</p>
```

Thêm CSS trực tiếp vào phần tử HTML bằng thuộc tính style



Internal CSS

<style> tag in the HTML <head> section

```
<style>
  p {
    color: red;
  }
</style>
```

Dùng thẻ <style> trong phần <head> của HTML



External CSS

Link a separate CSS file with <link>

```
<link rel="stylesheet"
      href="style.css">
```

Liên kết một file CSS riêng bằng thẻ <link>

CSS Selectors

Bộ chọn trong CSS



Element Selector

```
element { }
```

Targets HTML elements directly — affects all matching elements

```
p {  
  color: red;  
}
```

Chọn theo tên thẻ; áp dụng cho mọi thẻ khớp



ID Selector

```
#idname { }
```

Targets a unique element by ID — must be unique on a page

```
#header {  
  color: blue;  
}
```

Chọn một phần tử duy nhất theo id



Class Selector

```
.classname { }
```

Targets elements by class name — reusable for many elements

```
.button {  
  background: orange;  
}
```

Chọn theo tên class; dùng lại nhiều lần

Combining Selectors

Kết hợp các bộ chọn



Used to select related elements

Combine selectors to target elements based on their relationship in the HTML structure.

Dùng để chọn các phần tử có quan hệ. Ví dụ `div > strong` chọn `<strong>` là con trực tiếp của `<div>`.

```
div > strong {  
  color: tomato;  
}
```

CSS Combining Selectors

Day 3 · CSS Selectors

- Element Selector
- ID Selector
- Class Selector

CSS Box Model

Mô hình hộp trong CSS

Content

The actual content area where text and images appear

Vùng nội dung thật, nơi chứa chữ và hình ảnh

Padding

Space between content and border, inside the element

Khoảng cách giữa nội dung và viền, nằm bên trong phần tử

Border

Line around the padding, can have color and width

Đường viền bao quanh padding, có thể đặt màu và độ dày

Margin

Space outside the border, separates elements from each other

Khoảng cách bên ngoài viền, tách các phần tử ra khỏi nhau

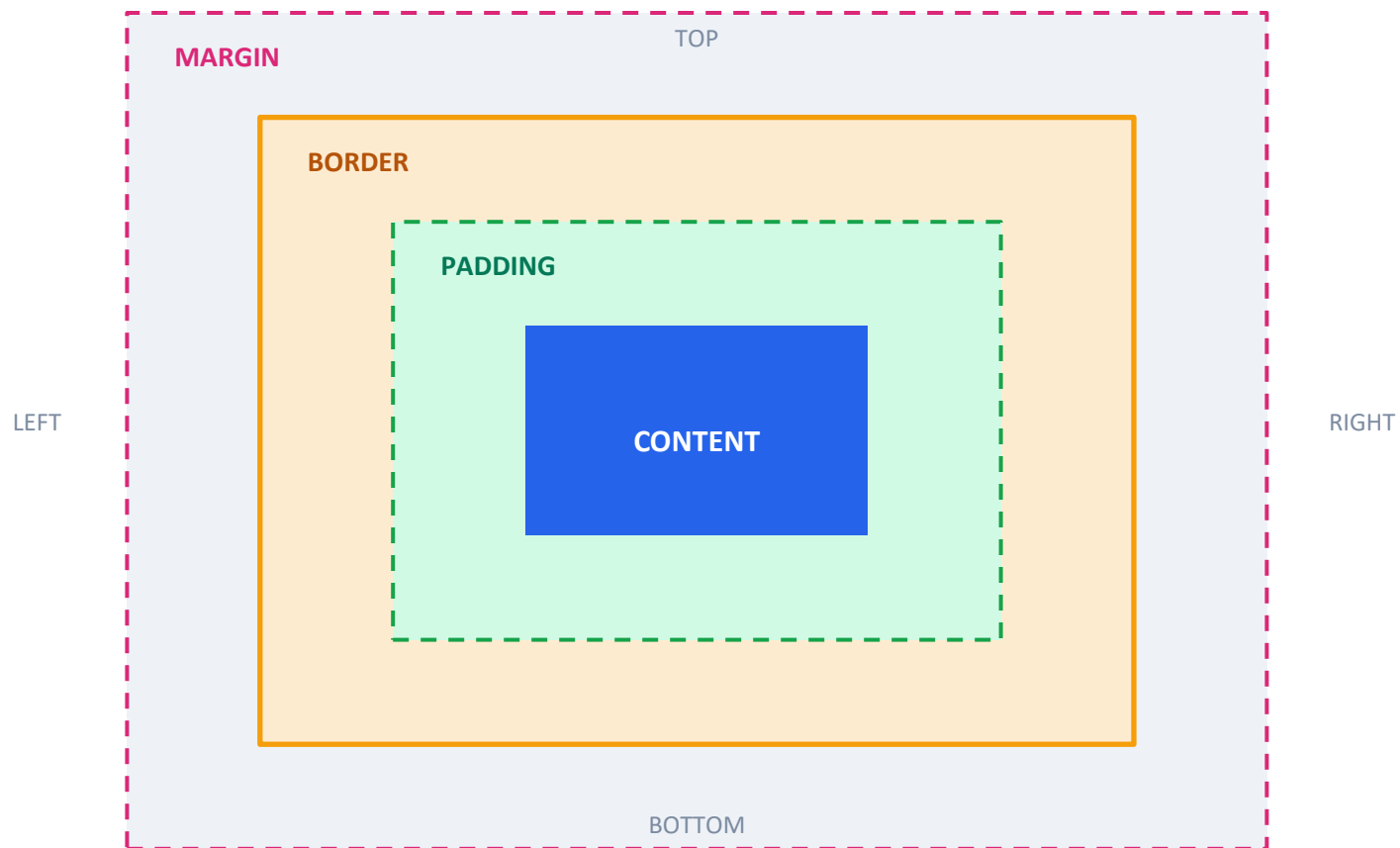
Box-sizing

Property controlling how width and height are calculated

Thuộc tính quyết định cách tính chiều rộng và chiều cao

CSS Box Model

Sơ đồ hộp: đọc từ trong ra ngoài



Đọc từ trong ra ngoài: Nội dung → Padding (đệm) → Border (viền) → Margin (lề ngoài cùng).

Afternoon Session Overview

1

Color Styling

color, hex, rgb/rgba

2

Font Styling

family, size, weight, align

3

Background Styling

color, image, size, repeat

4

Hands-on Practice

apply & experiment

Buổi chiều: tạo màu (color), tạo kiểu chữ (font), tạo nền (background), và phần thực hành.

Color Styling in CSS

Tạo màu trong CSS



Color Names

Easy to use — 140+ predefined colors

```
p {  
  color: red;  
}
```

Dễ dùng — có hơn 140 tên màu định sẵn



Hex Codes

Uses hexadecimal values — precise color control

```
p {  
  color: #ff5733;  
}
```

Dùng giá trị thập lục phân — kiểm soát màu chính xác



RGB & RGBA Values

RGB: color values · RGBA: adds transparency

```
p {  
  color:  
    rgba(255,0,0,.5);  
}
```

RGB: giá trị màu — RGBA: thêm độ trong suốt

Font Styling Properties

Các thuộc tính tạo kiểu chữ



Font-family

Defines the typeface used for text

*Xác định kiểu chữ (phông) dùng
cho văn bản*



Font-size & Weight

Controls the size and boldness of text

*Điều chỉnh cỡ chữ và độ đậm của
chữ*



Text-align & Line-height

Aligns text and sets line spacing

*Căn chữ theo chiều ngang và
khoảng cách dòng*



Text Decoration

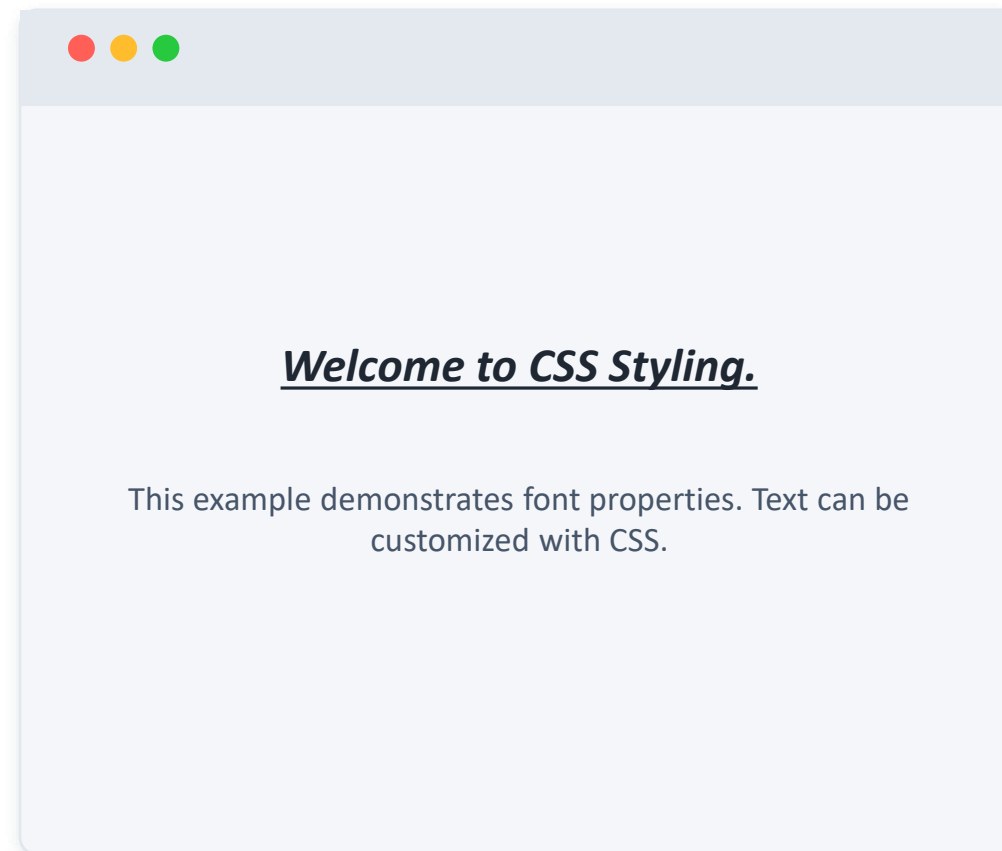
Underline, line-through and more

*Trang trí chữ: gạch chân, gạch
ngang...*

Font Styling Example

Ví dụ tạo kiểu chữ

```
<style>
  p {
    font-family: Arial, sans-serif;
    font-size: 20px;
    font-weight: bold;
    text-align: center;
    line-height: 1.8;
    text-decoration: underline;
  }
</style>
```



Ví dụ: đoạn văn được in đậm, gạch chân, căn giữa và giãn dòng rộng hơn nhờ các thuộc tính chữ.

Background Styling Properties

Các thuộc tính tạo nền



Background-color

Sets the background color of an element

```
background-color: #fff;
```

Đặt màu nền cho phần tử



Background-image

Adds an image as the background

```
background-image: url("img.jpg");
```

Thêm một hình ảnh làm nền



Size & Position

Controls image size and placement

```
background-size: cover;
```

Điều chỉnh kích thước và vị trí ảnh nền



Background-repeat

Controls whether the image repeats

```
background-repeat: no-repeat;
```

Quyết định ảnh nền có lặp lại hay không

Background Styling Example

Ví dụ tạo nền

```
<style>
  body {
    background-color: #4a90d9;
    background-image:
      url("bg.jpg");
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
  }
</style>
```



Ví dụ: nền có màu, ảnh phủ kín (cover), canh giữa (center), và không lặp lại (no-repeat).

CSS Practice

Thực hành CSS



Apply CSS

- Create a CSS file
- Connect it to HTML

Gắn CSS vào trang HTML; tạo file CSS và kết nối với HTML



Style Text

- Change colors
- Modify fonts

Tùy chỉnh giao diện chữ; đổi màu, đổi phông chữ



Create Layouts

- Add margins & padding
- Apply borders

Xây dựng bố cục trang; thêm lề & đệm, áp dụng viền



Experiment

- Use background colors
- Add background images

Thử nghiệm với nền; dùng màu nền và thêm ảnh nền

NEXT...

CSS Layout

Build Responsive Web Pages with Flexbox and Media Queries

Flexbox · Responsive Design · Media Queries

Buổi sau: Bố cục CSS — Flexbox, thiết kế responsive và Media Queries để trang web tự thích ứng mọi màn hình.

WEB PROGRAMMING · DA NANG IT

CSS Layout

Flexbox & Responsive Design

DAY 4 · 5 hours · Bố cục CSS — Flexbox & Thiết kế responsive

What We Covered Yesterday

Những gì chúng ta đã học hôm qua (CSS cơ bản)

1

CSS Connection

Inline · Internal ·
External

Cách gắn CSS vào HTML

2

Selectors

element · #id · .class

Bộ chọn — chọn phần tử để tạo kiểu

3

Box Model

content · padding ·
border · margin

Mô hình hộp của mỗi phần tử

4

Styling

color · font ·
background

Tạo kiểu màu, chữ, nền

Today we make those pages arrange and adapt.

Hôm nay chúng ta sắp xếp và làm trang tự thích ứng.

TODAY

Table of Contents

Mục lục — Ngày 4

1

MORNING

Why Layout?

Tại sao cần bố cục — cách phần tử tự sắp xếp

3

MORNING

Aligning Items

justify-content · align-items · gap

5

AFTERNOON

Media Queries

Đổi giao diện theo kích thước màn hình

2

MORNING

Flexbox Basics

Hộp linh hoạt: container & items, trục chính/phụ

4

AFTERNOON

Responsive Design

Thiết kế thích ứng nhiều màn hình

6

AFTERNOON

Layout Project

Thực hành tổng hợp: trang responsive

MORNING SESSION · 2 HOURS

Flexbox

The modern way to arrange elements in a row or column — and align them with ease.

Buổi sáng: Flexbox — cách hiện đại để sắp xếp phần tử theo hàng hoặc cột, và căn chỉnh chúng dễ dàng.

What is Flexbox

Direction & axes

justify / align

Wrap, gap & practice

Why Do We Need Layout?

Tại sao cần bố cục?

By default, HTML elements stack in a fixed flow — hard to place side by side.

Mặc định, các phần tử HTML xếp chồng theo một dòng cố định — khó đặt cạnh nhau.

Default flow

block element

block element

block element

display controls flow

Thuộc tính display quyết định cách phần tử dàn dòng.

block

Takes full width, stacks vertically

Chiếm cả dòng, xếp dọc

inline

Sits in line with text

Nằm chung dòng với chữ

flex

Lays children in a flexible row/column

Sắp các phần tử con linh hoạt

What is Flexbox?

Flexbox là gì?

A flex container arranges its child items along a main axis, with a perpendicular cross axis.

Một flex container sắp xếp các phần tử con dọc theo trục chính (main axis), với trục phụ (cross axis) vuông góc.



```
.container {  
  display: flex;  
}  
/* children become  
flex items */
```

Container = parent · Items = children. The parent gets `display:flex`; its direct children become flex items.

Container = cha (parent) · Items = các con (children)

flex-direction

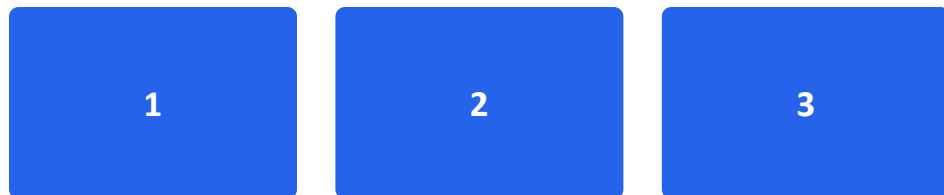
Hướng sắp xếp

Sets the main axis: a row (horizontal) or a column (vertical).

Đặt trục chính: theo hàng ngang (row) hoặc theo cột dọc (column).

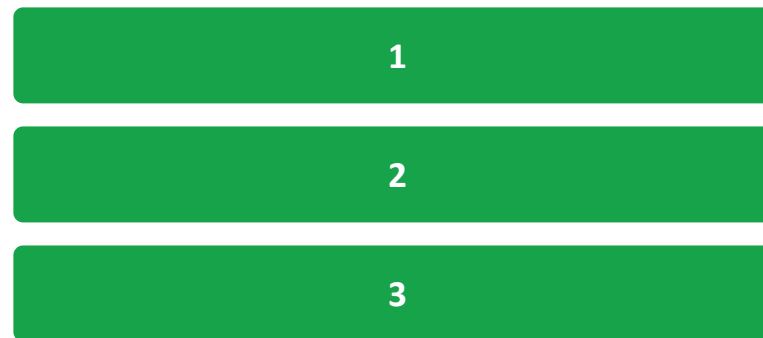
flex-direction: row;

(default · mặc định)



→ left to right / trái sang phải

flex-direction: column;



↓ top to bottom / trên xuống dưới

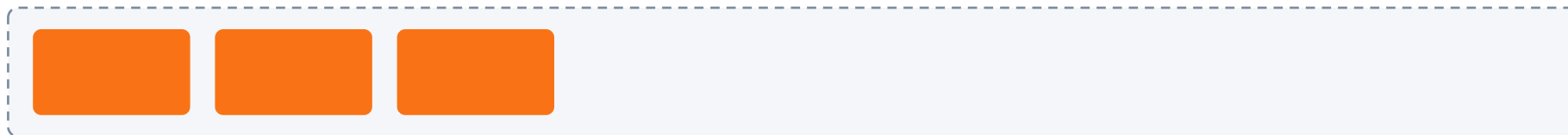
justify-content

Căn theo trục chính

Aligns items along the main axis · Căn các phần tử dọc theo trục chính.

flex-start

gom về đầu



center

gom vào giữa



space-between

cách đều, sát 2 mép



space-around

cách đều xung quanh

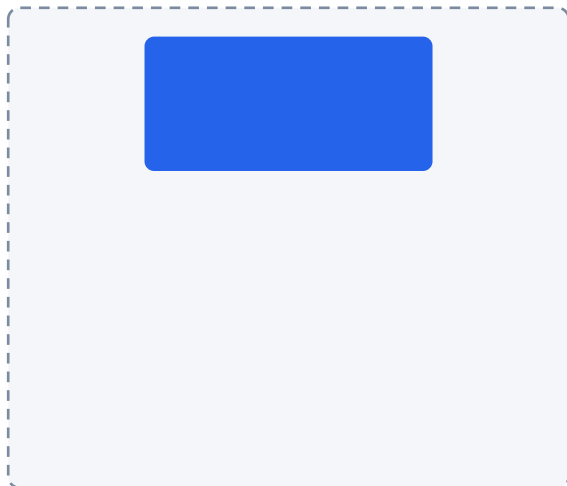


align-items

Căn theo trục phụ

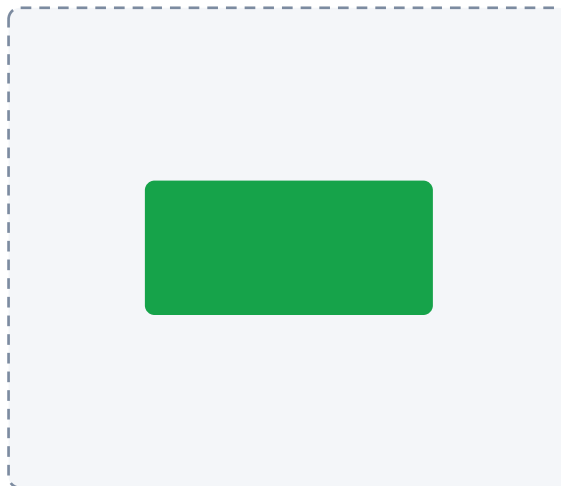
Aligns items along the cross axis · Căn các phần tử dọc theo trục phụ.

flex-start



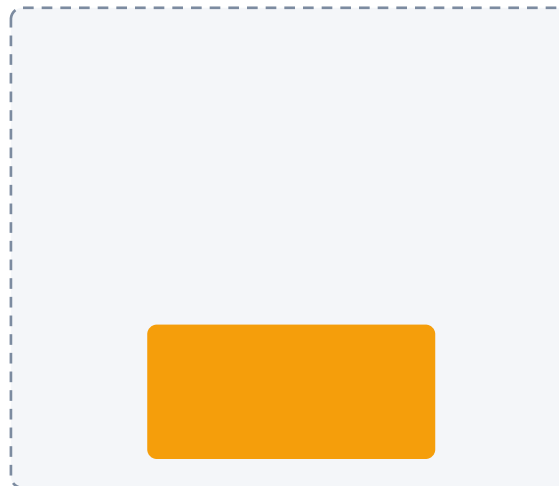
căn lên trên

center



căn vào giữa

flex-end



căn xuống dưới

stretch



kéo giãn đầy chiều cao

Tip: combine justify-content + align-items to center anything both ways.

Kết hợp cả hai để căn giữa hoàn toàn.

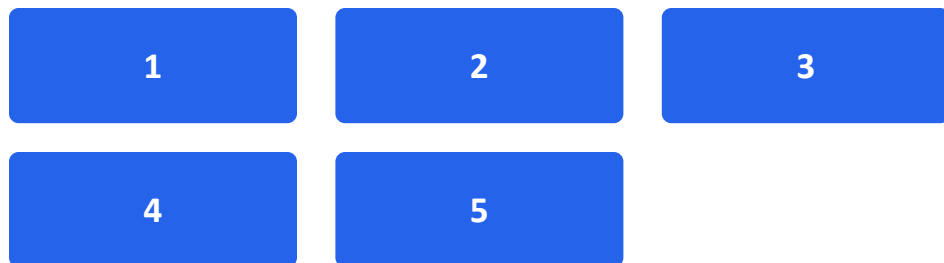
flex-wrap & gap

Xuống dòng & khoảng cách

flex-wrap: wrap;

Items move to a new line when they run out of room.

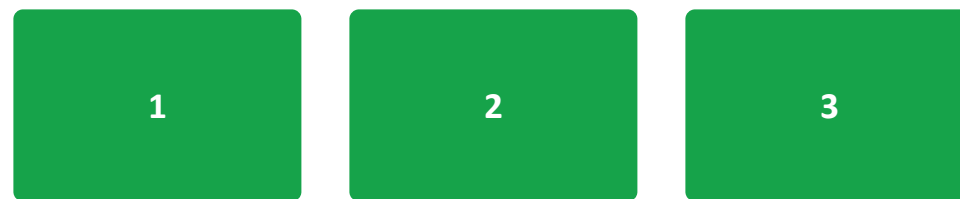
Phần tử tự xuống dòng khi hết chỗ.



gap: 16px;

Even spacing between items, no margins.

Khoảng cách đều giữa các phần tử — khỏi cần margin.



```
.container { display: flex; flex-wrap: wrap; gap: 16px; }
```

Practice: Build with Flexbox

Thực hành: dựng thanh điều hướng và hàng thẻ bằng Flexbox

1

Navbar row

Logo on the left, menu links pushed to the right with justify-content: space-between.

Thanh điều hướng: logo bên trái, link đẩy sang phải

2

Card row

Three cards side by side, equal gaps, that wrap on small widths.

Ba thẻ ngang nhau, cách đều, tự xuống dòng

3

Center a box

Use justify-content + align-items to perfectly center one box.

Căn giữa hoàn toàn một hộp

~45 min · Pair up & compare layouts / Ghép cặp và so sánh kết quả

AFTERNOON SESSION • 3 HOURS

Responsive Design

Make one page look great on phones, tablets, and desktops.

Buổi chiều: Thiết kế responsive — một trang hiển thị đẹp trên điện thoại, máy tính bảng và máy tính.

Responsive basics

Media queries

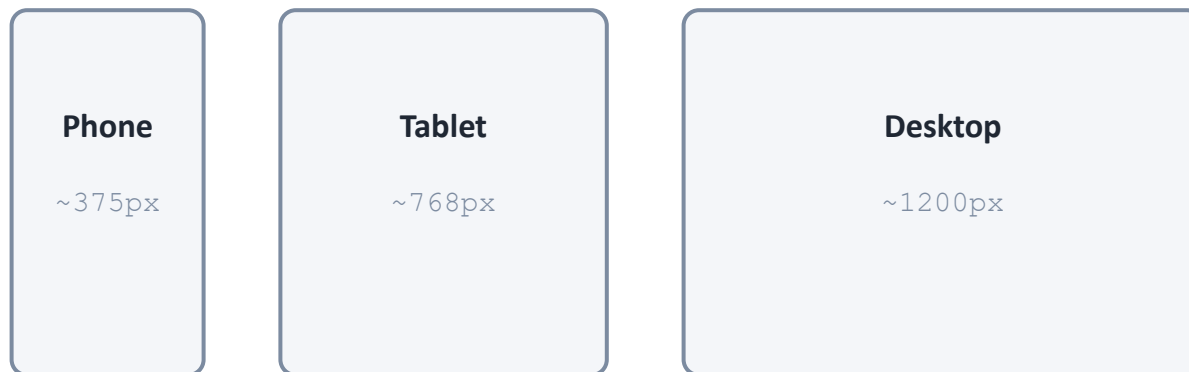
Comprehensive project

Responsive Design

Thiết kế thích ứng

One layout that adapts to any screen size, instead of a fixed pixel design.

Một bố cục tự thích ứng với mọi kích thước màn hình, thay vì cố định theo pixel.



- **Viewport meta** *Khai báo viewport*
- **Relative units** *Dùng đơn vị %, vw, rem*
- **Mobile-first** *Ưu tiên di động trước*

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Media Queries

Truy vấn phương tiện

Apply CSS only when a condition (like screen width) is true.

Áp dụng CSS chỉ khi một điều kiện (như chiều rộng màn hình) đúng.

```
/* phones: under 768px */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

Common breakpoints · Điểm ngắt

- **Mobile** < 768px
- **Tablet** 768–1024px
- **Desktop** > 1024px

Inside the @media block, override styles for that screen size. max-width = up to this size · min-width = from this size up

Trong khối @media, ghi đè kiểu cho kích thước màn hình đó.

Example: Cards that Stack

Ví dụ: thẻ xếp chồng

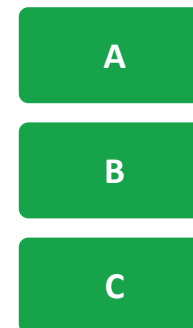
On desktop the cards sit in a row; on mobile a media query switches them to a column.

Trên desktop các thẻ nằm ngang; trên mobile, media query chuyển chúng thành cột.

Desktop · flex row



Mobile · flex column



```
@media (max-width: 768px) { .cards { flex-direction: column; } }
```

Comprehensive Layout Project

Thực hành tổng hợp: dựng một trang responsive hoàn chỉnh

1

Header

Flex navbar with logo + links

Thanh điều hướng bằng flex

2

Hero

Centered title & button

Khu giới thiệu căn giữa

3

Card grid

Flex-wrap cards with gap

Lưới thẻ dùng flex-wrap + gap

4

Responsive

Stack on mobile via @media

Xếp dọc trên mobile

~90 min · Build it together, step by step / Cùng dựng từng bước

Key Takeaways

Tóm tắt những điểm chính của ngày 4

- ✓ **display: flex** turns a parent into a flex container
biến phần tử cha thành flex container
- ✓ **direction + justify + align** control flow and alignment
điều khiển hướng và căn chỉnh
- ✓ **gap & flex-wrap** — spacing and wrapping made easy
khoảng cách và xuống dòng dễ dàng
- ✓ **@media queries** adapt the layout per screen size
điều chỉnh bố cục theo màn hình

NEXT · DAY 5

Mini Project ①

Build your own self-introduction web page with HTML + CSS.

Buổi sau: Tự làm trang web giới thiệu bản thân bằng HTML + CSS, rồi nhận góp ý theo nhóm.

Cảm ơn các bạn! · See you tomorrow.

DAY 5 · MINI PROJECT ①

Build Your Self-Introduction Page

Trang web giới thiệu bản thân — HTML + CSS

Putting everything together: structure with HTML, style with CSS.

From Day 1 to Day 4

1

HTML Structure

Tags, headings, semantic layout.

Thẻ, tiêu đề, bố cục ngữ nghĩa.

2

HTML Content

Forms, tables, images & video.

Biểu mẫu, bảng, hình ảnh & video.

3

CSS Basics

Selectors, box model, colors, fonts.

Bộ chọn, box model, màu, phong chữ.

4

CSS Layout

Flexbox & responsive design.

Flexbox & thiết kế responsive.

Hôm nay: kết hợp tất cả để tự xây một trang web hoàn chỉnh của riêng bạn.

What You'll Build Today

A one-page personal website

Trang web cá nhân một trang giới thiệu chính bạn

- **Introduce who you are**
Giới thiệu bạn là ai
- **Show your skills & interests**
Thể hiện kỹ năng & sở thích
- **Style it with your own colors**
Tạo kiểu với màu sắc của riêng bạn
- **Share & get team feedback**
Chia sẻ & nhận phản hồi từ nhóm

Today's flow

- | | |
|---------|--|
| ~30 min | Plan & sketch
<i>Lên kế hoạch</i> |
| ~90 min | Build HTML + CSS
<i>Xây HTML + CSS</i> |
| ~30 min | Polish & check
<i>Hoàn thiện</i> |
| ~30 min | Team feedback
<i>Phản hồi nhóm</i> |

Required Sections

1

Header

Your name + a short tagline.

Tên bạn + một câu giới thiệu ngắn.

2

About Me

A short paragraph about yourself.

Một đoạn văn ngắn về bản thân.

3

Skills / Hobbies

A list of what you can do or enjoy.

Danh sách kỹ năng hoặc sở thích.

4

Photo or Avatar

An image that represents you.

Một hình ảnh đại diện cho bạn.

5

Contact

Email or social links.

Email hoặc liên kết mạng xã hội.

6

Your own style

Custom colors, fonts & spacing.

Màu, phông chữ & khoảng cách riêng.

Plan & Sketch First

Sketch your sections top to bottom

Phác thảo các phần từ trên xuống dưới trước khi viết code

Header / Name

About

Skills

Contact

Why plan first?

- **Know what tags you need**
Biết cần dùng thẻ nào
- **Avoid getting stuck halfway**
Tránh bị kẹt giữa chừng
- **Easier to style later**
Dễ tạo kiểu hơn về sau
- **A simple paper sketch is enough**
Một bản phác trên giấy là đủ

Build the Structure

```
<body>
  <header>
    <h1>Your Name</h1>
    <p>Future web developer</p>
  </header>
  <section id="about">
    <h2>About Me</h2>
    <p>I am a student who ...</p>
  </section>
  <section id="skills">
    <h2>My Skills</h2>
    <ul><li>HTML</li><li>CSS</li></ul>
  </section>
</body>
```

Use semantic tags

<header> for the top intro

Phần đầu trang

<section> to group content

Nhóm nội dung

<h1> ~ <h2> for titles

Tiêu đề

**** for lists

Danh sách

**** for your photo

Hình ảnh

Make It Yours with CSS

1 Colors

Background & text colors that fit you.

Màu nền & màu chữ hợp với bạn.

2 Fonts

Pick a font-family & sizes.

Chọn phong chữ & cỡ chữ.

3 Spacing

Use margin & padding to breathe.

Dùng margin & padding cho thoáng.

4 Layout

Center content, align with Flexbox.

Căn giữa, sắp xếp bằng Flexbox.

5 Images

Round corners, set width nicely.

Bo góc, đặt chiều rộng đẹp mắt.

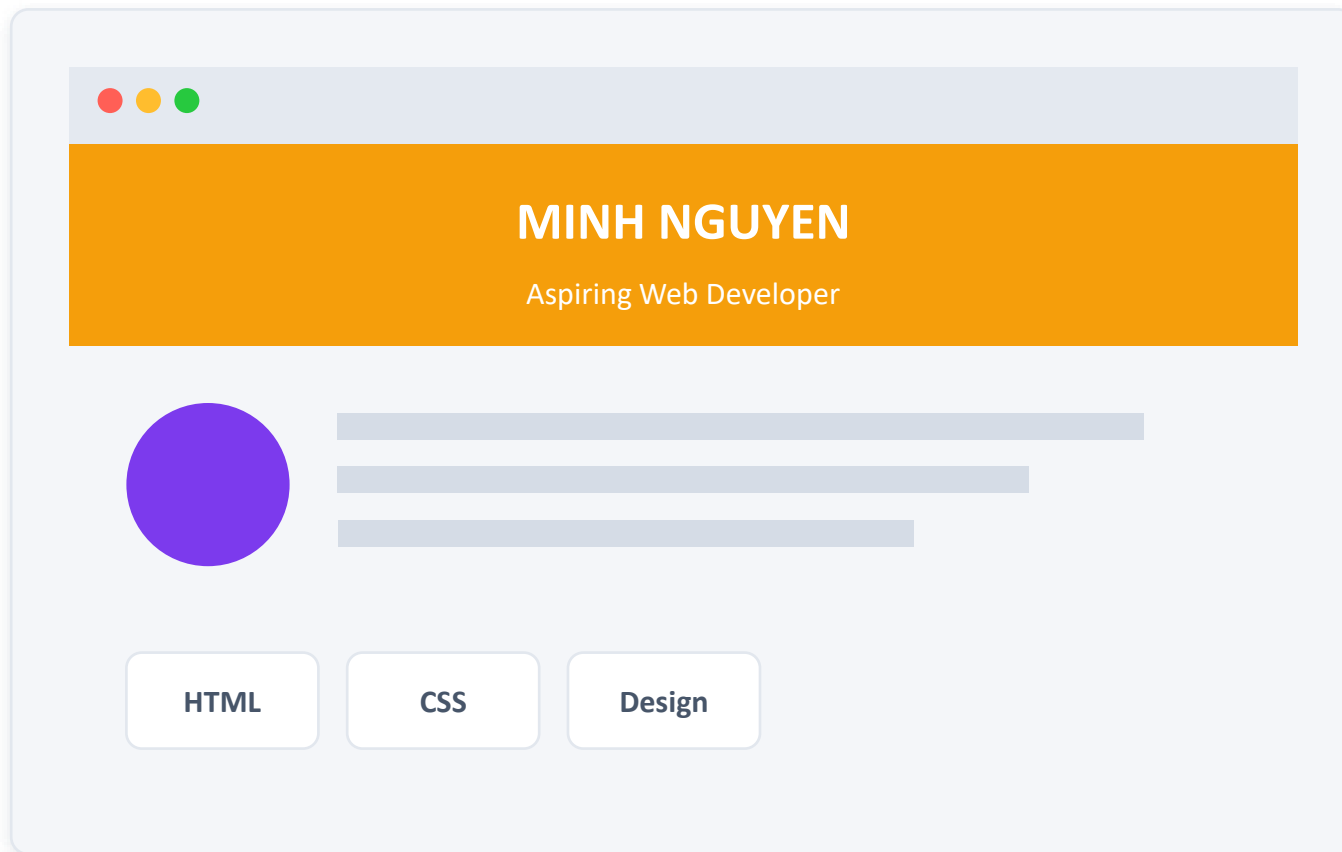
6 Hover

Add a hover effect on links.

Thêm hiệu ứng hover cho liên kết.

EXAMPLE

One Way It Could Look



This is just an idea

Đây chỉ là một gợi ý — hãy tự do sáng tạo theo phong cách của bạn!

- **Make it your own**
Hãy biến nó thành của bạn
- **Any colors you like**
Màu sắc tùy thích
- **Keep it readable**
Giữ cho dễ đọc

AVOID THESE

Common Mistakes & Tips

⚠ Common mistakes

✗ Forgetting to link the CSS file

Quên liên kết file CSS

✗ Unclosed tags </...>

Quên đóng thẻ </...>

✗ Wrong file path for images

Sai đường dẫn ảnh

✗ Too many colors at once

Quá nhiều màu một lúc

✓ Helpful tips

✓ Save & refresh often

Lưu & tải lại trang thường xuyên

✓ Use browser DevTools (F12)

Dùng DevTools của trình duyệt (F12)

✓ Build first, style after

Xây cấu trúc trước, tạo kiểu sau

✓ Ask & help your teammates

Hỏi & giúp đỡ bạn cùng nhóm

Team Feedback Time

Share your screen and present your page to the team. Be kind and specific.

Chia sẻ màn hình và giới thiệu trang của bạn với nhóm. Hãy góp ý tử tế và cụ thể.

1 What I like

One thing that looks great.

Một điểm trông rất đẹp.

2 One idea

One thing to improve.

Một điều có thể cải thiện.

3 One question

Ask how they did something.

Hỏi cách họ đã làm điều gì đó.

WRAP-UP • NEXT

Great work today!

Bạn vừa tự xây trang web đầu tiên của mình 🎉

Tomorrow: JavaScript — make your pages interactive.

Ngày mai: JavaScript — làm cho trang web có thể tương tác.

DAY 6

JavaScript Basics

Cơ bản về JavaScript

Variables • Data Types • Operators • Functions • Conditionals

Biến • Kiểu dữ liệu • Toán tử • Hàm • Câu lệnh điều kiện

Web Programming Team • IT Volunteer Program

JS

What is JavaScript?

JavaScript là gì?

JavaScript is a programming language that makes web pages interactive.

JavaScript là ngôn ngữ lập trình giúp trang web trở nên tương tác.

HTML

Structure — the skeleton

Cấu trúc — bộ khung

CSS

Style — the appearance

Kiểu dáng — vẻ ngoài

JS

Behavior — the actions

Hành vi — hành động



Analogy: A house. HTML is the frame, CSS is the paint and furniture, JavaScript is the electricity that makes lights and doors work.

Ví dụ: Một ngôi nhà. HTML là bộ khung, CSS là sơn và nội thất, JavaScript là điện làm cho đèn và cửa hoạt động.

HTML + CSS + JS Together

HTML + CSS + JS kết hợp

The three layers work as a team to build a complete web page.

Ba lớp này phối hợp như một đội để tạo nên trang web hoàn chỉnh.

JS

Behavior / Hành vi

CSS

Style / Kiểu dáng

HTML

Structure / Cấu trúc



```
<!-- HTML: structure -->
<button id="btn">Click</button>

/* CSS: style */
#btn { color: red; }

// JS: behavior
btn.onclick = function() {
  alert("Hi!");
};
```

PART 1

How to Add JavaScript

Cách thêm JavaScript

- **Internal script**
Mã nội bộ
- **External script**
Mã bên ngoài
- **Where to place <script>**
Vị trí đặt thẻ <script>



Internal Script

Mã nội bộ



```
<body>
  <h1>Hello</h1>
  <script>
    alert("Welcome!");
  </script>
</body>
```

Write JS inside HTML

Write JavaScript directly inside the HTML using a *Viết JavaScript trực tiếp trong HTML bằng thẻ <script>.* `<script>` tag.

✓ Good for quick tests

Tốt cho thử nghiệm nhanh

✗ Hard to reuse on many pages

Khó tái sử dụng trên nhiều trang



alert() shows a pop-up message in the browser.

alert() hiển thị một hộp thông báo trên trình duyệt.

External Script

Mã bên ngoài

Keep JavaScript in a separate .js file and link it with src.

Đặt JavaScript trong tệp .js riêng và liên kết bằng src.

index.html

```
<script src="app.js">
</script>
```

app.js

```
alert("From app.js!");
```

Why external?

- **Reuse the same code on many pages**

Tái sử dụng cùng một mã trên nhiều trang

- **Cleaner, easier to maintain**

Gọn gàng, dễ bảo trì hơn

Where to Place `<script>`

Đặt thẻ `<script>` ở đâu

Place `<script>` just before `</body>` so the HTML loads first.

Đặt `<script>` ngay trước `</body>` để HTML được tải trước.

✗ Risky — in `<head>`

```
<head>
  <script src="app.js">
</head>
// page not ready yet!
```

✓ Recommended — end of `<body>`

```
<h1>Hello</h1>
<script src="app.js">
</body>
// everything is ready
```



Analogy: read the recipe ingredients (HTML) before you start cooking (JS). If JS runs first, it looks for elements that don't exist yet.

Ví dụ: đọc nguyên liệu (HTML) trước khi nấu (JS). Nếu JS chạy trước, nó sẽ tìm các phần tử chưa tồn tại.

PART 2

Variables

Biến

- **let** — value can change
let — giá trị có thể thay đổi
- **const** — value stays fixed
const — giá trị cố định
- **var** — the old way
var — cách cũ



What is a Variable?

Biến là gì?

A variable is a named box used to store a piece of data.

Biến là một chiếc hộp có tên dùng để lưu trữ dữ liệu.

"Maria"

name

```
let name = "Maria";  
// keyword  name  value
```

Label + value

The box has a label (name) and holds a value ("Maria"). You read or change what's inside by its label.

Chiếc hộp có nhãn (name) và chứa giá trị ("Maria"). Bạn đọc hoặc thay đổi nội dung qua nhãn của nó.

let — Changeable Variable

let — biến có thể thay đổi



```
let age = 20;  
console.log(age); // 20  
age = 21; // changed!  
console.log(age); // 21
```

Use let when value may change

Like a whiteboard: you can erase the value and write a new one any time.

Giống bảng trắng: bạn có thể xóa giá trị và viết giá trị mới bất cứ lúc nào.



console.log() prints a value to the browser console — your main tool for checking results.

console.log() in giá trị ra console của trình duyệt — công cụ chính để kiểm tra kết quả.

const — Fixed Variable

const — biến cố định



```
const PI = 3.14;  
console.log(PI); // 3.14  
PI = 10;  
// Error: cannot reassign
```

Use const when it must not change

Like writing in permanent ink: once set, it cannot be erased or rewritten.

Giống như viết bằng mực không xóa được: đã đặt thì không thể xóa hay viết lại.



Tip: prefer const by default. Switch to let only when you know the value will change.

Mẹo: ưu tiên const. Chỉ dùng let khi bạn biết giá trị sẽ thay đổi.

var — The Old Way

var — cách cũ

var is the original keyword. It still works, but let and const are now preferred.

var là từ khóa ban đầu. Nó vẫn dùng được, nhưng nay let và const được ưa chuộng hơn.

Keyword / Từ khóa	Can change? / Thay đổi được?	Use today? / Dùng hôm nay?
let	Yes / Có	✓ Yes / Có
const	No / Không	✓ Yes (default) / Mặc định
var	Yes / Có	✗ Avoid / Nên tránh



Remember: const first, let when it changes, var almost never.

Ghi nhớ: const trước, let khi thay đổi, var hầu như không dùng.

PART 3

Data Types

Kiểu dữ liệu

- **String — text**
Chuỗi — văn bản
- **Number — numeric values**
Số — giá trị số
- **Boolean — true / false**
Boolean — đúng / sai
- **null & undefined — empty values**
null & undefined — giá trị rỗng



String — Text

Chuỗi — văn bản



```
let city = "Da Nang";
let hi = 'Xin chào';
// join with +
let msg = hi + " " + city;
// "Xin chào Da Nang"
```

Text in quotes

Use "double" or 'single' quotes — both work. Joining strings with + is called concatenation.

Dùng dấu "kép" hoặc 'đơn' đều được. Nối chuỗi bằng + gọi là phép nối chuỗi.



Note: numbers in quotes become text. "5" is a string, 5 is a number.

Lưu ý: số trong dấu ngoặc trở thành văn bản. "5" là chuỗi, 5 là số.

Number — Numeric Values

Số — giá trị số



```
let age = 20;  
let price = 9.99;  
// math works directly  
let total = age + 5; // 25
```

No quotes needed

A number can be a whole number or a decimal. No quotes needed.
Số có thể là số nguyên hoặc số thập phân. Không cần dấu ngoặc.



No quotes! Numbers can be added, subtracted, multiplied and divided directly.

Không có dấu ngoặc! Số có thể cộng, trừ, nhân, chia trực tiếp.

Boolean — true / false

Boolean — đúng / sai



```
let isStudent = true;  
let isRaining = false;  
// used to make decisions
```

Only two values

Like a light switch: it is either ON (true) or OFF (false), nothing in between.

Giống công tắc đèn: hoặc BẬT (true) hoặc TẮT (false), không có ở giữa.



Booleans power conditions — they decide which code runs (we'll see this with if statements).

Boolean điều khiển điều kiện — quyết định đoạn mã nào chạy (sẽ thấy ở câu lệnh if).

null & undefined — Empty Values

null & undefined — giá trị rỗng

Both mean "no value", but for different reasons.

Cả hai đều nghĩa là "không có giá trị", nhưng vì lý do khác nhau.

undefined

A box was created but nothing was put inside yet.

Hộp đã tạo nhưng chưa bỏ gì vào.

```
let x;  
// x is undefined
```

null

You purposely set the box to empty.

Bạn cố ý đặt hộp thành rỗng.

```
let y = null;  
// empty on purpose
```



undefined = the system left it empty. null = you chose to empty it.

undefined = hệ thống để trống. null = bạn chủ động làm trống.

PART 4

Operators

Toán tử

- **Arithmetic — math**
Số học — phép tính
- **Comparison — compare values**
So sánh — so sánh giá trị
- **Logical — combine conditions**
Logic — kết hợp điều kiện



Arithmetic Operators

Toán tử số học

Used for everyday math on numbers.

Dùng cho các phép toán hằng ngày trên số.

+

Add / Cộng

$$5 + 2 = 7$$

-

Subtract / Trừ

$$5 - 2 = 3$$

*

Multiply / Nhân

$$5 * 2 = 10$$

/

Divide / Chia

$$6 / 2 = 3$$

%

Remainder / Số dư

$$7 \% 2 = 1$$



% (modulo) gives the remainder after division. $7 \% 2 = 1$. It is handy for checking even/odd numbers: $\text{even} \% 2 = 0$.

% (modulo) cho phần dư sau khi chia. $7 \% 2 = 1$. Hữu ích để kiểm tra số chẵn/lẻ: $\text{số chẵn} \% 2 = 0$.

Comparison Operators

Toán tử so sánh

Compare two values. The result is always a boolean (true / false).

So sánh hai giá trị. Kết quả luôn là boolean (true / false).

Operator	Meaning / Ý nghĩa	Example	Result
<code>===</code>	Equal value & type / Bằng nhau	<code>5 === 5</code>	<code>true</code>
<code>!==</code>	Not equal / Khác nhau	<code>5 !== 3</code>	<code>true</code>
<code>></code>	Greater than / Lớn hơn	<code>7 > 10</code>	<code>false</code>
<code><</code>	Less than / Nhỏ hơn	<code>3 < 8</code>	<code>true</code>
<code>>=</code>	Greater or equal / Lớn hơn bằng	<code>5 >= 5</code>	<code>true</code>



Always use `===` (three equals) to compare. It checks value AND type, avoiding surprises.

Luôn dùng `===` (ba dấu bằng) để so sánh. Nó kiểm tra cả giá trị VÀ kiểu, tránh bất ngờ.

Logical Operators

Toán tử logic

Combine several conditions into one true / false result.

Kết hợp nhiều điều kiện thành một kết quả true / false.

& &

AND / VÀ

Both must be true

Cả hai phải đúng

```
(age>18 && hasID)
```

| |

OR / HOẶC

At least one is true

Ít nhất một đúng

```
(isSat || isSun)
```

!

NOT / PHỦ ĐỊNH

Reverses the value

Đảo ngược giá trị

```
!isRaining
```



Like a club entry: age over 18 AND has ID. Both true → enter. With OR, just one is enough.

Như vào câu lạc bộ: trên 18 tuổi VÀ có giấy tờ. Cả hai đúng → vào được. Với HOẶC, chỉ cần một là đủ.

What we have learned so far

Những gì chúng ta đã học



Adding JS

Thêm JS

internal, external, placement



Variables

Biến

let, const, var



Data Types

Kiểu dữ liệu

string, number, boolean, null



Operators

Toán tử

arithmetic, comparison, logical



Next, this afternoon → Functions & Conditionals

Tiếp theo, buổi chiều → Hàm và Câu lệnh điều kiện

PART 5

Functions

Hàm

- **Define a function**
Định nghĩa hàm
- **Parameters — inputs**
Tham số — đầu vào
- **Return value — output**
Giá trị trả về — đầu ra



Defining a Function

Định nghĩa hàm



```
function sayHello() {  
  console.log("Hello!");  
}  
sayHello(); // call -> Hello!
```

Reusable block of code

Like a coffee machine: build it once, then press the button to use it again and again.

Giống máy pha cà phê: làm một lần, rồi nhấn nút để dùng lại nhiều lần.



Define once, call many times. Calling uses the function name followed by ().

Định nghĩa một lần, gọi nhiều lần. Gọi hàm bằng tên hàm kèm theo ().

Parameters — Inputs

Tham số — đầu vào



```
function greet(name) {  
  console.log("Hi " + name);  
}  
greet("Linh"); // Hi Linh  
greet("Nam");  // Hi Nam
```

Different inputs each time

Like a coffee machine button for the flavor: the same machine makes different drinks depending on the input you give.

Giống nút chọn vị trên máy cà phê: cùng một máy tạo ra đồ uống khác nhau tùy đầu vào bạn đưa.

Return Value — Output

Giá trị trả về — đầu ra



```
function add(a, b) {  
  return a + b;  
}  
let sum = add(3, 4);  
console.log(sum); // 7
```

return hands back a result

return gửi kết quả tính toán về bạn lưu trữ hoặc dùng lại.



console.log shows a value on screen. return hands the value back to your code so it can be used again.

console.log hiển thị giá trị ra màn hình. return trả giá trị về cho mã của bạn để dùng tiếp.

Conditionals

Câu lệnh điều kiện

- **if — run when true**
if — chạy khi đúng
- **if / else — choose between two**
if / else — chọn giữa hai
- **else if — multiple paths**
else if — nhiều nhánh



if — Run When True

if — chạy khi đúng



```
let age = 20;  
if (age >= 18) {  
  console.log("Adult");  
} // runs -> Adult
```

Runs only when true

Like an umbrella rule: IF it is raining, take an umbrella.
If not, do nothing.

Như quy tắc ô dù: NẾU trời mưa, mang theo ô. Nếu không, không làm gì.



The condition in () must result in true or false — this is where comparison operators are used.

Điều kiện trong () phải cho ra true hoặc false — đây là nơi dùng toán tử so sánh.

if / else — Two Paths

if / else — hai nhánh



```
let age = 15;  
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
} // -> Minor
```

An alternative path

A fork in the road: one way IF the condition is true, the other way ELSE. You always take exactly one path.

Ngã ba đường: một hướng NẾU điều kiện đúng, hướng còn lại nếu KHÔNG. Luôn đi đúng một hướng.

else if — Multiple Paths

else if — nhiều nhánh



```
let score = 85;
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B"); // -> B
} else { console.log("C"); }
```

First true one runs

Like grading a test: check the highest grade first, then work down. JS stops at the first match.

Như chấm điểm: kiểm tra mức cao nhất trước, rồi giảm dần. JS dừng ở điều kiện khớp đầu tiên.

PRACTICE 1

Age Checker Program

Chương trình kiểm tra độ tuổi

Task / Nhiệm vụ

Given an age, print "Adult" if 18 or older, otherwise print "Minor".

Cho một độ tuổi, in "Adult" nếu từ 18 trở lên, ngược lại in "Minor".

Hint: use a variable for age, then if / else with the >= operator.

Gợi ý: dùng một biến cho tuổi, rồi if / else với toán tử >=.

Solution / Lời giải



```
let age = 20;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

// Output: Adult

PRACTICE 2

Score Grade Program

Chương trình xếp loại điểm số

Task / Nhiệm vụ

Print a grade from a score: 90+ = A, 80–89 = B, 70–79 = C, else = F

In ra xếp loại từ điểm số: 90+ = A, 80–89 = B, 70–79 = C, else = F

Hint: use if / else if / else. Check the highest score first.

Gợi ý: dùng if / else if / else. Kiểm tra điểm cao nhất trước.

Solution / Lời giải



```
let score = 85;
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 70) {
  console.log("C");
} else { console.log("F"); }
```

// Output: B

WRAP-UP

Great work today!

Làm tốt lắm hôm nay!

- JavaScript adds behavior to web pages
- Store data with let and const
- Use operators to calculate and compare
- Functions reuse code; conditionals make decisions

JavaScript thêm hành vi cho trang web

Lưu dữ liệu bằng let và const

Dùng toán tử để tính và so sánh

Hàm tái sử dụng mã; điều kiện đưa ra quyết định

Tomorrow → JavaScript Events: loops, arrays, and the DOM

Ngày mai → Sự kiện JavaScript: vòng lặp, mảng và DOM

Loops, Arrays & Events

Vòng lặp, Mảng và Sự kiện — làm trang web phản hồi người dùng

Morning: loops & arrays · Afternoon: DOM & event handlers

JS

Today's Roadmap

Lộ trình hôm nay

1

Loops — repeat work automatically

Vòng lặp — lặp lại công việc tự động

AM

2

Arrays — store many values together

Mảng — lưu nhiều giá trị cùng nhau

AM

3

DOM — control the page with JS

DOM — điều khiển trang bằng JS

PM

4

Events — react to clicks & input

Sự kiện — phản ứng khi nhấp & nhập

PM

Loops

Vòng lặp — lặp lại công việc mà không lặp lại mã



What is a Loop?

Vòng lặp là gì?

A loop repeats a block of code many times.

Vòng lặp lặp lại một khối mã nhiều lần.

Instead of writing the same line 100 times, you write it once.

Thay vì viết cùng một dòng 100 lần, bạn chỉ viết một lần.

Think of a music player on "repeat" — same song, again and again.

Hãy nghĩ đến nút "lặp lại" trên trình phát nhạc — cùng bài hát, lặp đi lặp lại.



```
// without a loop
console.log("Hi");
console.log("Hi");
// ...97 more times

// with a loop
for (let i = 0; i < 100; i++) {
  console.log("Hi");
}
```

The for Loop — Anatomy

Cấu trúc vòng lặp for



```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```



1. Start → `let i = 0`

Khởi tạo — điểm bắt đầu



2. Condition → `i < 5`

Điều kiện — chạy khi còn đúng



3. Update → `i++`

Cập nhật — tăng sau mỗi vòng

Output / Kết quả:

0 1 2 3 4

`i` goes from 0 to 4 — five times total.

`i` chạy từ 0 đến 4 — tổng cộng 5 lần.

How a for Loop Runs

Vòng lặp for chạy như thế nào

1

Run the start ($i = 0$)

Chạy phần khởi tạo

2

Check the condition

Kiểm tra điều kiện

3

If true → run the body

Nếu đúng → chạy thân vòng lặp

4

Run the update ($i++$)

Chạy phần cập nhật

↺

Go back to step 2

Quay lại bước 2

**When the condition becomes false,
the loop stops.**

Khi điều kiện trở thành sai, vòng lặp dừng lại.

break & continue

break và continue



```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  console.log(i);  
}  
// prints 0 1 2 3 4
```

Two control keywords

- **break — stop the loop completely**
break — dừng hẳn vòng lặp
- **continue — skip to the next round**
continue — bỏ qua, sang vòng tiếp theo
- **Used more later**
(Giới thiệu ngắn — sẽ dùng nhiều hơn sau này)

Arrays

Mảng — lưu nhiều giá trị trong một biến



What is an Array?

Mảng là gì?

An array stores many values in one variable.

Mảng lưu nhiều giá trị trong một biến.

- Like an egg carton: one box, many slots.

Giống ví trứng: một hộp, nhiều ô.

- Each slot has a number called an index.

Mỗi ô có một số gọi là chỉ số (index).

- Indexes start at 0, not 1.

Chỉ số bắt đầu từ 0, không phải 1.



```
let fruits = [  
  "apple", "banana", "kiwi"  
];
```

apple

index 0

banana

index 1

kiwi

index 2

Access, Modify & Length

Truy cập, sửa đổi & độ dài



```
let fruits = ["apple", "kiwi"];  
fruits[0]; // "apple"  
fruits[1] = "mango";  
fruits.length; // 2
```

Remember

- **fruits[0] → first item**
phần tử đầu tiên
- **fruits[length-1] → last item**
phần tử cuối cùng
- **.length → item count**
.length cho biết số phần tử



Use the index in [] to read or change a value.

Dùng chỉ số trong [] để đọc hoặc đổi giá trị.

push & pop

push và pop



```
let list = ["a", "b"];  
list.push("c");  
// ["a", "b", "c"]  
list.pop();  
// ["a", "b"]
```

Add / remove at end

- **push()** — add an item to the end

push() — thêm phần tử vào cuối

- **pop()** — remove the last item

pop() — xóa phần tử cuối cùng

- **Like a stack of plates**

Giống chồng đĩa: thêm/lấy từ trên cùng

Arrays + Loops

Mảng + Vòng lặp

A loop can visit every item in an array.

Vòng lặp có thể duyệt qua từng phần tử của mảng.

Use *i* as the index, run until *i* < length.

Dùng i làm chỉ số, chạy đến khi i < length.



```
let names = ["An", "Bo", "Ci"];  
for (let i=0; i<names.length; i++){  
  console.log(names[i]);  
}
```

Output / Kết quả

An

Bo

Ci

Sum & Average

Tổng & Trung bình



```
let scores = [80, 90, 100];  
let total = 0;  
for (let i=0; i<scores.length; i++){  
  total += scores[i];  
} // total = 270, avg = 90
```

Running total

- **Keep a running total in the loop**

Cộng dồn vào một biến tổng

- **Divide by .length for average**

Chia cho .length để có trung bình

PRACTICE ①

Times Table

Thực hành ① — Bảng cửu chương

Task / Nhiệm vụ

Print the 3 times table (3×1 ... 3×9) using a for loop.

In bảng nhân 3 (3×1 ... 3×9) bằng vòng lặp for.

*Hint: Loop i from 1 to 9, print $3 * i$.*

*Gợi ý: Lặp i từ 1 đến 9, in $3 * i$.*

solution.js



```
for (let i=1; i<=9; i++){  
  console.log(  
    "3 x " + i + " = " + (3*i)  
  );  
}
```


PRACTICE ②

Array Sum

Thực hành ② — Tổng mảng

Task / Nhiệm vụ

Given [10, 20, 30, 40], find the total and the average.

Cho [10, 20, 30, 40], tính tổng và trung bình.

Hint: total += arr[i], then total / arr.length.

Gợi ý: total += arr[i], rồi total / arr.length.

solution.js



```
let a = [10, 20, 30, 40];
let total = 0;
for(let i=0; i<a.length; i++)
    total += a[i];
let avg = total / a.length;
// total = 100, avg = 25
```

What we have learned

Ôn tập buổi sáng

- **for loops repeat code automatically**
Vòng lặp for lặp lại mã tự động
- **Array indexes start at 0**
Chỉ số mảng bắt đầu từ 0
- **push() adds, pop() removes from the end**
push() thêm, pop() xóa ở cuối
- **Loop + array = visit every item**
Vòng lặp + mảng = duyệt mọi phần tử

The DOM

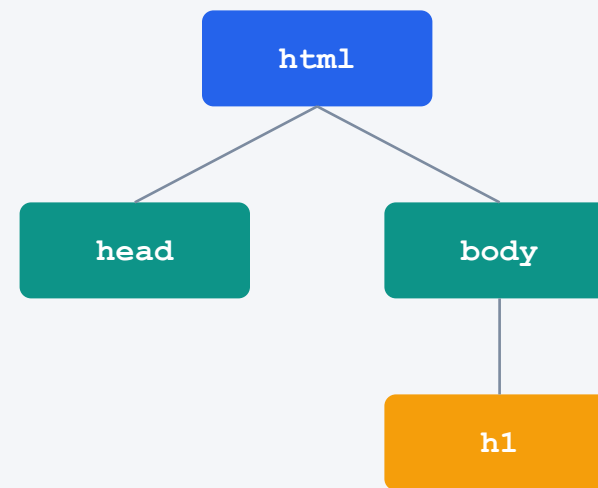
DOM — điều khiển HTML bằng JavaScript



What is the DOM?

DOM là gì?

- **DOM = Document Object Model.**
DOM = Mô hình Đối tượng Tài liệu.
- **The browser turns your HTML into a tree of objects.**
Trình duyệt biến HTML thành một cây các đối tượng.
- **JavaScript can read and change that tree.**
JavaScript có thể đọc và thay đổi cây đó.
- **= JS can change the page after it loads.**
= JS có thể đổi trang sau khi tải.



Selecting Elements (1)

Chọn phần tử (1)



```
<h1 id="title">Hi</h1>
```

```
let el = document.getElementById(  
  "title"  
);
```

Grab by id

- **getElementById finds one element by id**

tìm một phần tử theo id

- **document = the whole page object**

Toàn bộ trang dưới dạng đối tượng

- **id must be unique on the page**

id phải là duy nhất trên trang

Selecting Elements (2)

Chọn phần tử (2)



```
// by class .box
document.querySelector(".box");

// by tag, all of them
document.querySelectorAll("button");
```

Same selectors as CSS!

- **querySelector uses CSS selectors**
querySelector dùng bộ chọn CSS
- **querySelectorAll returns every match**
querySelectorAll trả về mọi kết quả
- **#id, .class, tag — like CSS**
Cùng bộ chọn như CSS

Events

Sự kiện — phản ứng với hành động của người dùng



What is an Event?

Sự kiện là gì?

- **An event is something the user does.**
Sự kiện là điều người dùng thực hiện.
- **Click, type, move the mouse, press a key...**
Nhấp, gõ, di chuyển chuột, nhấn phím...
- **Like a doorbell: someone presses → it rings.**
Giống chuông cửa: ai đó nhấn → nó reo.
- **JS can listen and respond.**
JS có thể lắng nghe và phản hồi.

click

nhấp chuột

input

nhập liệu

submit

gửi biểu mẫu

addEventListener

addEventListener



```
let btn = document.getElementById("go");  
btn.addEventListener("click", function() {  
    alert("Clicked!");  
});
```

Pattern

● **element.addEventListener(...)**

Bảo phần tử lắng nghe một sự kiện

● **"event", function**

Khi xảy ra, chạy một hàm

Reading Input Values

Đọc giá trị ô nhập



```
<input id="name">
```

```
let box = document.getElementById("name");  
let text = box.value;  
console.log(text);
```

Use .value

- **Get what the user typed with .value**

Lấy nội dung người dùng gõ bằng .value

- **Read it inside the event function**

Đọc nó bên trong hàm sự kiện

PRACTICE ③

Click to Change

Thực hành ③ — Nhấp để thay đổi

Task / Nhiệm vụ

When the button is clicked, change the heading text.

Khi nhấn nút, đổi nội dung tiêu đề.

Hint: Select both, add a click listener, set .textContent.

Gợi ý: Chọn cả hai, thêm trình nghe click, đặt .textContent.

solution.js



```
let h = document.getElementById("t");
let b = document.getElementById("go");
b.addEventListener("click", ()=>{
  h.textContent = "Changed!";
});
```

DAY 7 · WRAP-UP

Great work today!

Làm tốt lắm hôm nay!

Loops · Arrays · DOM · Events

Vòng lặp · Mảng · DOM · Sự kiện

Next — Day 8: DOM manipulation & building a mini app

JS

Changing the Page with JS

Thay đổi trang web bằng JS — thêm, sửa, xóa & xây ứng dụng nhỏ

Morning: edit the DOM · Afternoon: build a mini app

JS

Today's Roadmap

Lộ trình hôm nay

1

Edit content & style of elements

Sửa nội dung & kiểu của phần tử

AM

2

Create and remove elements

Tạo và xóa phần tử

AM

3

Build a To-do / Quiz app

Xây ứng dụng To-do / Quiz

PM

4

Test & debug your app

Kiểm thử & sửa lỗi ứng dụng

PM

PART 1 · MORNING

Change Elements

Sửa đổi phần tử — nội dung, kiểu & thuộc tính



Quick Recap — Selecting

Ôn nhanh — Chọn phần tử



```
// by id
document.getElementById("box");

// by CSS selector
document.querySelector(".box");
```

Select first, then change

- **Yesterday: we found elements**
Hôm qua: chúng ta đã tìm phần tử
- **Today: we change them**
Hôm nay: chúng ta thay đổi chúng
- **Step 1 is always: select first**
Bước 1 luôn là: chọn phần tử trước

Change Text — `textContent`

Đổi văn bản — `textContent`



```
let h = document.getElementById("t");  
h.textContent = "Hello!";  
// the heading now shows: Hello!
```

Set the inner text

- **`.textContent` reads or sets text**

đọc hoặc đặt văn bản bên trong

- **Safe for plain text**

An toàn cho văn bản thuần

Change HTML — innerHTML

Đổi HTML — innerHTML



```
box.innerHTML =  
  "<b>Bold</b> text";
```

Insert real tags

- **.innerHTML sets HTML, not just text**

đặt HTML, không chỉ văn bản

- **Insert tags like or **

*Có thể chèn thẻ như hoặc *

- **⚠ Don't put user input directly**

Không đưa dữ liệu người dùng trực tiếp

Change Look — style

Đổi giao diện — style



```
box.style.color = "red";  
box.style.fontSize = "24px";  
box.style.backgroundColor  
  = "#F7DF1E";
```

CSS from JS

- **.style changes CSS from JavaScript**
đổi CSS từ JavaScript
- **Property names use camelCase**
Tên thuộc tính dùng camelCase
- **background-color → backgroundColor**
background-color → backgroundColor

Toggle CSS — classList

Bật/tắt lớp CSS — classList



```
box.classList.add("active");  
box.classList.remove("active");  
box.classList.toggle("active");
```

Switch a class

- **Better: define styles in CSS, switch class**

định nghĩa kiểu trong CSS, đổi lớp

- **add / remove / toggle a class**

add / remove / toggle một lớp

Change Attributes

Đổi thuộc tính



```
img.setAttribute("src", "cat.png");
```

Any HTML attribute

- **setAttribute** changes any attribute

đổi bất kỳ thuộc tính HTML nào

- **e.g. src** of an image, **href** of a link

vd: src của ảnh, href của liên kết

PART 2 · MORNING

Add & Remove

Thêm & Xóa phần tử



Create & Add Elements

Tạo & Thêm phần tử



```
let li = document.createElement  
  ("li");  
li.textContent = "New item";  
list.appendChild(li);
```

Three steps

- **1. createElement makes a new tag**
tạo thẻ mới
- **2. set its text or class**
đặt văn bản hoặc lớp
- **3. appendChild puts it on the page**
đưa nó lên trang

Remove Elements

Xóa phần tử



```
let item = document.getElementById("x");  
item.remove();
```

Delete an element

- **.remove() deletes an element**
xóa một phần tử
- **Often called from a delete button**
Thường gọi từ nút xóa

Add a List Item

Thực hành — Thêm mục danh sách

Task / Nhiệm vụ

Click a button to add a new `<li>` to a list.

Nhấn nút để thêm `<li>` mới vào danh sách.

Hint: create → textContent → appendChild, inside a click listener.

Gợi ý: tạo → textContent → appendChild, trong trình nghe click.

solution.js



```
btn.addEventListener("click", ()=>{  
  let li = document.createElement("li");  
  li.textContent = "Item";  
  list.appendChild(li);  
});
```

What we have learned

Ôn tập buổi sáng

- **textContent changes text; innerHTML changes HTML**

textContent đổi văn bản; innerHTML đổi HTML

- **style uses camelCase; classList toggles classes**

style dùng camelCase; classList đổi lớp

- **createElement + appendChild = add to page**

createElement + appendChild = thêm vào trang

- **.remove() deletes an element**

.remove() xóa một phần tử

PART 3 · AFTERNOON

Build a Mini App

Xây ứng dụng nhỏ — To-do hoặc Quiz



Plan First

Lập kế hoạch trước

Before coding, decide what your app does. Pick one:

Trước khi code, quyết định ứng dụng làm gì. Chọn một:



To-do List

Danh sách việc cần làm

Add tasks, mark done, delete.

Thêm việc, đánh dấu xong, xóa.



Quiz App

Ứng dụng đố vui

Show a question, check the answer, score.

Hiện câu hỏi, kiểm tra đáp án, tính điểm.

To-do App — Features

Ứng dụng To-do — Chức năng

1

Type a task and click Add

Gõ việc và nhấn Thêm

2

New task appears in the list

Việc mới xuất hiện trong danh sách

3

Click a task to mark it done

Nhấp vào việc để đánh dấu xong

4

Click ✕ to delete it

Nhấn ✕ để xóa

Step 1 — HTML Skeleton

Bước 1 — Khung HTML



```
<input id="task">
<button id="add">Add</button>
<ul id="list"></ul>
```

3 ids to remember

- **task → the input**
ô nhập
- **add → the button**
nút bấm
- **list → the **
danh sách

Step 2 — Add a Task

Bước 2 — Thêm việc

On click, read the input and create an `<li>`.

Khi nhấp, đọc ô nhập và tạo `<li>`.

`app.js`

```
let input = document.getElementById("task"),
    list = document.getElementById("list");
document.getElementById("add").addEventListener("click", () => {
  let li = document.createElement("li");
  li.textContent = input.value;
  list.appendChild(li);
});
```

Step 3 — Delete & Done

Bước 3 — Xóa & Hoàn thành

Click an item to toggle a "done" class. A ✕ button removes it.

Nhấp vào việc để bật/tắt lớp "done". Nút ✕ để xóa.

app.js



```
li.addEventListener("click", ()=> li.classList.toggle("done"));  
let x = document.createElement("button");  
x.textContent = "✕";  
x.addEventListener("click", ()=> li.remove());
```


Variant — Quiz App

Biến thể — Ứng dụng Quiz

Same tools, different idea. Compare the answer, update a score.

Cùng công cụ, ý tưởng khác. So sánh đáp án, cập nhật điểm.

quiz.js



```
let score = 0;
btn.addEventListener("click", ()=>{
  if (input.value === "4") score++;
  result.textContent = "Score: " + score;
});
```

Test & Debug

Kiểm thử & Sửa lỗi

- **Open the browser Console (F12) for errors**

Mở Console trình duyệt (F12) để xem lỗi

- **Use `console.log()` to check values**

Dùng `console.log()` để kiểm tra giá trị

- **Empty input? Don't add a blank task**

Ô nhập rỗng? Đừng thêm việc trống

- **Test every button: add, done, delete**

Thử mọi nút: thêm, xong, xóa

DAY 8 · WRAP-UP

You built an app!

Bạn đã xây một ứng dụng!

Edit · Add · Remove · Build

Sửa · Thêm · Xóa · Xây dựng

Next — Day 9: your own free-topic mini project

JS

DAY 9 • MINI PROJECT ②

Build Your Own Project

Xây dự án của riêng bạn — từ ý tưởng đến sản phẩm

Morning: plan & structure • Afternoon: style & code

JS

Project Overview

Tổng quan dự án

1

Goal — make a small working web page with HTML + CSS + JS

Mục tiêu — tạo trang web nhỏ chạy được bằng HTML + CSS + JS

2

In teams — free topic of your choice

Theo nhóm — chủ đề tự do

3

Today: plan & build · Tomorrow: present

Hôm nay: lập kế hoạch & xây · Ngày mai: trình bày

PART 1 · MORNING

Plan & Structure

Lập kế hoạch & dựng khung



Pick a Topic

Chọn chủ đề

Keep it small and finishable today. Some ideas:

Giữ nhỏ và làm xong được hôm nay. Một vài ý tưởng:



Guestbook

Sổ lưu bút

Leave & show messages

Để lại & hiển thị tin nhắn



Calculator

Máy tính

Add, subtract, show result

Cộng, trừ, hiện kết quả



Number Game

Trò chơi số

Guess a random number

Đoán số ngẫu nhiên

Plan Before Coding

Lập kế hoạch trước khi code



① Feature list

Danh sách chức năng



Write down what the app should do.

Viết ra những gì ứng dụng cần làm.



Start with the most important one.

Bắt đầu với chức năng quan trọng nhất.



② Screen sketch

Phác thảo màn hình



Draw the layout on paper first.

Vẽ bố cục ra giấy trước.



Where do inputs, buttons, output go?

Đặt ô nhập, nút, kết quả ở đâu?

Write the HTML Structure

Viết khung HTML



```
<header>My Project</header>
<main>
  <input id="in">
  <button id="go">Go</button>
  <div id="out"></div>
</main>
```

Skeleton first



Build with semantic tags

Đựng khung bằng thẻ ngữ nghĩa



Give ids to elements JS will use

Đặt id cho phần tử mà JS sẽ dùng

By lunch you should have

Mốc buổi sáng — trước giờ nghỉ trưa

- **A chosen topic and feature list**

Chủ đề và danh sách chức năng

- **A rough screen sketch**

Bản phác thảo màn hình

- **The HTML skeleton in place**

Khung HTML đã dựng xong

PART 2 · AFTERNOON

Style & Code

Tạo kiểu & lập trình



Style with CSS

Tạo kiểu bằng CSS



```
main{  
  display: flex;  
  gap: 12px;  
}  
.done { color: gray; }
```

Reuse Days 3-4

- **Reuse what you learned on Days 3-4**

Dùng lại kiến thức ngày 3-4

- **Flexbox for layout, colors & fonts**

Flexbox cho bố cục, màu & phông

Add JS Features

Thêm chức năng JS



```
let go = document.getElementById("go");
go.addEventListener("click", ()=>{
  let v = document.getElementById("in").value;
  out.textContent = v;
});
```

Reuse Days 6-8



Connect events to actions

Kết nối sự kiện với hành động



Use what you built in Days 6-8

Dùng những gì đã làm ngày 6-8

Build One Step at a Time

Xây từng bước một

1

Make the most important feature work first

Làm chức năng quan trọng nhất chạy trước

2

Test it, then add the next feature

Kiểm thử, rồi thêm chức năng tiếp theo

3

Style it nicely once it works

Tạo kiểu đẹp khi đã chạy được

Check before you finish

Kiểm tra trước khi kết thúc

- **Every button and input works**

Mọi nút và ô nhập hoạt động

- **No errors in the Console (F12)**

Không có lỗi trong Console (F12)

- **Layout looks clean on screen**

Bố cục gọn gàng trên màn hình

- **Ready to explain it tomorrow**

Sẵn sàng giải thích vào ngày mai

DAY 9 · WRAP-UP

Almost there!

Sắp xong rồi!

Plan · Structure · Style · Code

Kế hoạch · Khung · Kiểu · Mã

Tomorrow — Day 10: present your project & celebrate

JS

DAY 10 • PRESENT & CELEBRATE

Show & Celebrate

Trình bày & Ăn mừng — chặng cuối của hành trình

Team presentations • Course review • Completion

JS

Today's Plan

Kế hoạch hôm nay

1

Each team presents their project

Mỗi nhóm trình bày dự án

2

Look back at the 10-day journey

Nhìn lại hành trình 10 ngày

3

Reflection & course completion

Suy ngẫm & hoàn thành khóa học

How to Present

Cách trình bày

Keep it short — about 3 minutes per team. Cover three things:

Ngắn gọn — khoảng 3 phút mỗi nhóm. Nói ba điều:

1

What you built

Bạn đã làm gì

The idea & main features

Ý tưởng & chức năng chính

2

Live demo

Trình diễn trực tiếp

Show it working

Cho xem nó chạy

3

What you learned

Bạn học được gì

Hardest part & a win

Phần khó nhất & điều làm được

Before you present

Trước khi trình bày

- **Open your project and test the demo once**
Mở dự án và thử demo một lần
- **Decide who says what**
Quyết định ai nói phần nào
- **Speak slowly and clearly — it's okay to be nervous**
Nói chậm và rõ — hồi hộp là bình thường

Team Presentations

Phần trình bày của các nhóm — chúc các bạn tự tin!

Audience: listen, then give one kind comment + one question.

Khán giả: lắng nghe, rồi một lời khen + một câu hỏi.

Our 10-Day Journey

Hành trình 10 ngày của chúng ta



From an empty page to a working app — look how far you came.

Từ trang trắng đến ứng dụng chạy được — bạn đã tiến rất xa.

What you can do now

Bây giờ bạn có thể làm gì

- **Structure a page with HTML**
Dựng trang bằng HTML
- **Style and lay it out with CSS**
Tạo kiểu & bố cục bằng CSS
- **Make it interactive with JavaScript**
Làm trang tương tác bằng JavaScript
- **Build a small app from scratch**
Xây ứng dụng nhỏ từ đầu

Keep Learning

Tiếp tục học



Practice

Luyện tập

Rebuild today's app alone

Tự làm lại ứng dụng hôm nay



Free resources

Tài nguyên miễn phí

MDN, freeCodeCamp

MDN, freeCodeCamp

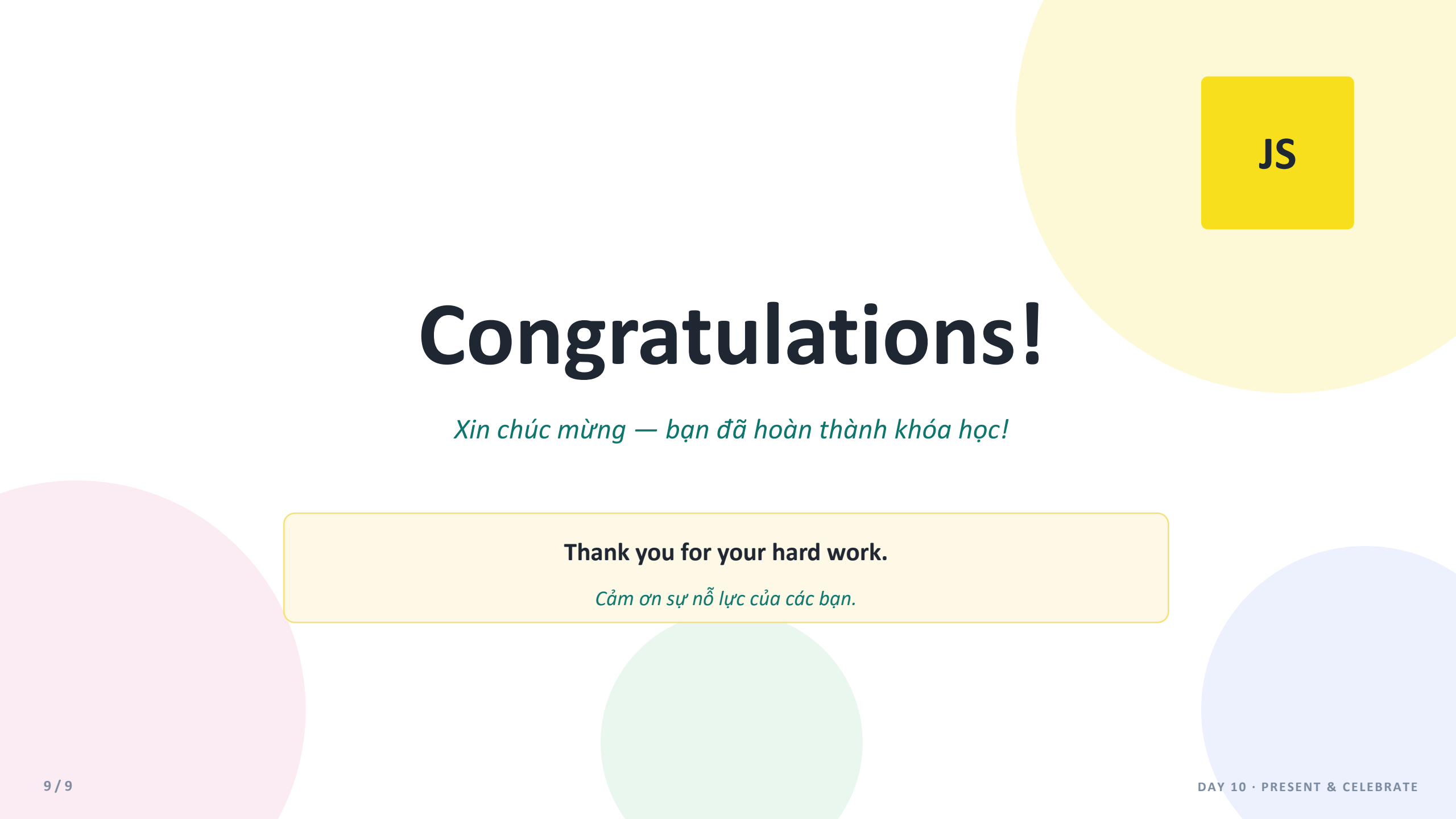


Build ideas

Xây ý tưởng

Make things you find useful

Làm thứ bạn thấy hữu ích



JS

Congratulations!

Xin chúc mừng — bạn đã hoàn thành khóa học!

Thank you for your hard work.

Cảm ơn sự nỗ lực của các bạn.